Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

INGENIERÍA EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL

Portfolio digital de la materia: Fundamentos de Machine learning

Equipo: Alt F4

Grado: Tercero Grupo: R

Indice de las investigaciones

INVESTIGACIÓN DE OUTLIERS ,CUARTILES, PUNTUACIÓN Z, ESTANDARIZAR Y NORMALIZAR 4

OUTLIERS 5

CUARTILES 6

PUNTUACIÓN Z 7

ESTANDARIZAR 8

NORMALIZAR 10

REFERENCIAS: 11

INVESTIGACIÓN KNN, ONE-HOT ENCODING Y LABEL ENCODING 12

1. Introducción 13

2. KNN (K-Nearest Neighbors) 13

3. One-Hot Encoding 14

4. Label Encoding 14

5. Diferencias entre las técnicas 15

6. Cómo realizarlo en Excel paso a paso 15

6.1 Preparar los datos 15

6.2 One-Hot Encoding en Excel 16

6.3 Label Encoding en Excel 16

6.4 KNN en Excel paso a paso 17

7. Programación en Python 18

7.1 Instalación 18

7.2 One-Hot Encoding con Python 18

7.3 Label Encoding con Python 19

7.4 KNN completo con One-Hot Encoding 19

8. Ejemplo de predicción 20

9. ¿Qué técnica conviene utilizar? 20

10. Conclusión 21

FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109

70 Aniversario Facultad de Ingeniería Tampico 1956-2026

11. Referencias 21

DATA LEAK 22

Referencias: 26

Errores Mae, Mse, Rmse, Mape 27

Referencias: 41

Matriz de confusion

Indice de las documentaciones

Documentacion de los codigos en clase

Documentación del código de regresión mulltivariada

Documentación del codigo de regresión logistica

Documentación del código de regresión logística multiclase (One-vs-Rest)

VERDAD, BELLEZA, PROBIDAD

INVESTIGACIÓN DE OUTLIERS ,CUARTILES, PUNTUACIÓN Z, ESTANDARIZAR Y NORMALIZAR

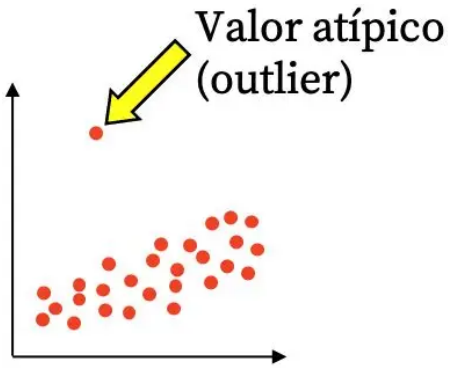
Carrera: Ingeniería en ciencia de datos he inteligencia artificial

Equipo: Alt F4

OUTLIERS

¿Qué son los valores atípicos (outliers)?

En estadística, los valores atípicos, también llamados observaciones atípicas u outliers, Es decir, un valor atípico es un valor anormal y que es extremadamente diferente al resto de valores de la muestra. No es necesariamente un error pero distorsiona la media y el análisis y hay que decir si se elimina o se conserva.



En **Excel**, una forma confiable de detectar outliers (valores atípicos) es usando el método del Rango Intercuartílico (IQR).

Método IQR en Excel

El IQR se calcula así: IQR $= Q3 - Q1$

Donde:

- **Q1** = Primer cuartil (25 % de los datos)
- **Q3** = Tercer cuartil (75 % de los datos)

Un valor es outlier si:

- **Menor que:** $Q1 - 1{,}5 \times IQR$
- **Mayor que:** $Q3 + 1{,}5 \times IQR$

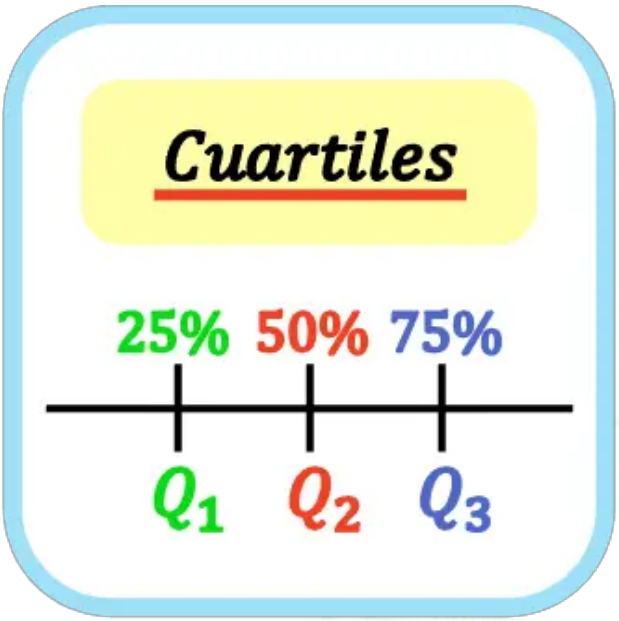
CUARTILES

En la página anterior se mencionaron los cuartiles pero ¿Qué son los cuartiles?

En estadística, los cuartiles son los tres valores que dividen a un conjunto de datos ordenados en cuatro partes iguales. Por lo tanto, el primer, segundo y tercer cuartil representan respectivamente el 25 %, 50 % y 75 % del conjunto de datos estadísticos.

FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109

Los cuartiles se representan mediante una Q mayúscula y el subíndice del cuartil, de manera que el primer cuartil es Q_1, el segundo cuartil es Q_2, y el tercer cuartil es Q_3.

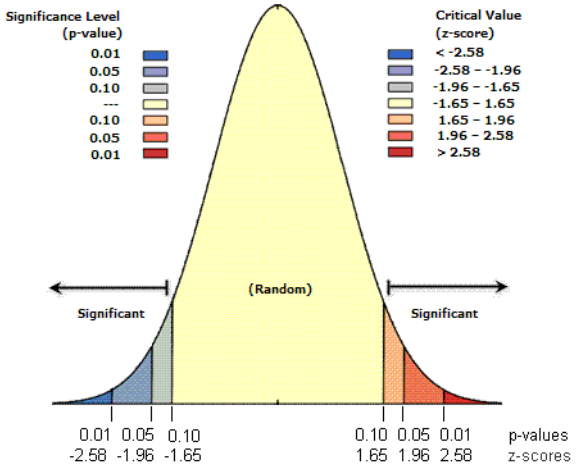


PUNTUACIÓN Z

¿Qué es la puntuación Z?

La **puntuación Z**, o **puntaje Z**, es una puntuación estadística que indica a cuántas desviaciones estándar de la media se encuentra un valor. Para calcular una puntuación Z de un valor se debe restar la media a dicho valor y luego dividir por la desviación estándar de la muestra de datos.

Por ejemplo, si un valor se encuentra dos desviaciones estándar por debajo de la media aritmética del conjunto de datos, la puntuación Z de ese valor es igual a -2.



En Excel, este cálculo puede realizarse de diferentes maneras. Una forma consiste en hacerlo paso a paso: primero se obtiene la media mediante la función PROMEDIO, después la desviación estándar con DESVEST.P si se trabaja con toda la población o con DESVEST.M si se trata de una muestra, y finalmente se aplica la fórmula $(A2 - \mu)/\sigma$para cada valor de la serie.

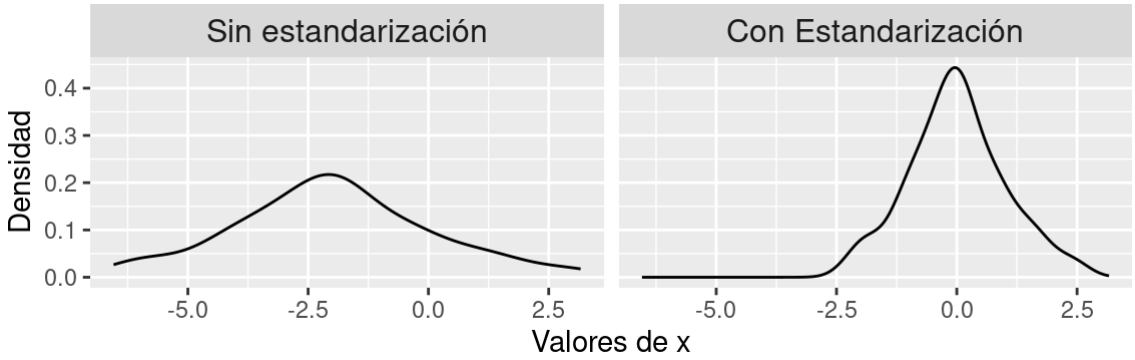
Otra alternativa es emplear una fórmula directa que combine todos los cálculos en una sola expresión, evitando el uso de celdas auxiliares. Por ejemplo, para una población se utiliza: =(A2 - PROMEDIO($A$2:$A$11)) / DESVEST.P($A$2:$A$11) y para una muestra se reemplaza por DESVEST.M.

Excel también ofrece la función integrada ESTANDARIZAR, que simplifica el procedimiento al calcular automáticamente el Z-score de cada dato a partir de la media y la desviación estándar del rango seleccionado.

Es fundamental tener presente que un Z-score positivo indica que el valor se encuentra por encima de la media, mientras que un Z-score negativo señala que está por debajo. Asimismo, la elección entre DESVEST.P y DESVEST.M depende de si se dispone de toda la población o únicamente de una muestra.

ESTANDARIZAR

La estandarización es una técnica estadística que se utiliza para transformar los datos en un formato común, lo que permite una comparación y un análisis más sencillos. Este proceso implica ajustar los valores de un conjunto de datos para que tengan una media de cero y una desviación estándar de uno.



¿Cómo aplicar la estandarización en Excel?

1. Coloca los datos que deseas estandarizar en una columna, por ejemplo, de A2:A11.
2. Calcula la media de los datos con:
   =PROMEDIO(A2:A11)
3. Calcula la desviación estándar:
   - Para una población: =DESVEST.P(A2:A11)
   - Para una muestra: =DESVEST.M(A2:A11)
4. En una nueva columna, aplica la fórmula de puntuación Z:
   =(A2-PROMEDIO($A$2:$A$11))/DESVEST.M($A$2:$A$11)
5. Presiona Enter y arrastra la fórmula hacia abajo para obtener la estandarización de todos los datos.

También puedes utilizar directamente la función “ESTANDARIZAR” de Excel:

=ESTANDARIZAR(A2,PROMEDIO($A$2:$A$11),DESVEST.M($A$2:$A$11))

De esta manera, cada dato se transforma a su valor estandarizado (Z), permitiendo comparar los datos independientemente de su escala original.

NORMALIZAR

En el análisis de datos, la **normalización estadística** es una técnica fundamental que permite transformar diferentes variables para que puedan ser comparadas y utilizadas de manera uniforme. La normalización es un proceso que ajusta los valores de los datos a una escala común, sin distorsionar las diferencias en los rangos de valores.

Estandarización	Normalización
$x_{stand} = \frac{x - mean(x)}{Standard\ Dev(x)}$	$x_{norm} = \frac{x - \min(x)}{\max(x) - \min(x)}$

¿Cómo aplicar la normalización en Excel?

1. Coloca los datos en una columna, por ejemplo, **A2:A11**.
2. En una nueva columna, escribe la fórmula:

=(A2-MIN($A$2:$A$11))/(MAX($A$2:$A$11)-MIN($A$2:$A$11))

3. Presiona **Enter**.
4. Arrastra la fórmula hacia abajo para aplicarla a todos los datos.

Con esto:

- El **valor mínimo** se convierte en **0**.
- El **valor máximo** se convierte en **1**.
- Los demás valores quedan entre **0 y 1**, manteniendo la proporción entre ellos.

REFERENCIAS:

▷ Valores atípicos (outliers): qué son, ejemplos, calculadora,...

▷ Cuartiles: qué son, fórmula, ejemplos, calculadora...

▷ Puntuación Z (estadística)

Normalización Estadística: Conceptos, Métodos y Aplicaciones | Estudyando

Qué es: Estandarizar - Entender la estandarización de datos

INVESTIGACIÓN KNN, ONE-HOT ENCODING Y LABEL ENCODING

Carrera: Ingeniería en ciencia de datos he inteligencia artificial

Equipo: Alt F4

1. Introducción

En aprendizaje automático es común trabajar con datos que contienen categorías, como sexo, ciudad, tipo de producto o sistema operativo. Muchos algoritmos necesitan que las variables estén representadas numéricamente. El KNN (K-Nearest Neighbors) es un algoritmo de aprendizaje supervisado que clasifica un dato según las clases de sus vecinos más cercanos. Para utilizar variables categóricas con algoritmos como KNN, es necesario aplicar técnicas de codificación como One-Hot Encoding o Label Encoding.

2. KNN (K-Nearest Neighbors)

KNN significa K-Nearest Neighbors o K vecinos más cercanos. Es un algoritmo de aprendizaje supervisado utilizado principalmente para clasificación y también para regresión. Para clasificar una observación, calcula su distancia respecto de ejemplos conocidos, selecciona los K vecinos más cercanos y, en clasificación, asigna la clase que tenga mayor representación entre ellos.

La distancia euclidiana entre dos puntos puede calcularse como:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + \ldots + (x_n - x'_n)^2}$$

El valor K debe elegirse con cuidado. Un K pequeño puede hacer al modelo sensible al ruido, mientras que un K demasiado grande puede suavizar demasiado las diferencias entre clases. En scikit-learn, KNeighborsClassifier implementa la votación de los K vecinos más cercanos.

3. One-Hot Encoding

One-Hot Encoding convierte una variable categórica en varias columnas binarias. Se crea una columna para cada categoría y se coloca 1 cuando la observación pertenece a esa categoría y 0 cuando no pertenece. Es especialmente útil cuando las categorías no tienen un orden natural, porque evita que el modelo interprete que una categoría es numéricamente mayor o menor que otra.

Ejemplo:

Color	Rojo	Azul	Verde
Rojo	1	0	0
Azul	0	1	0
Verde	0	0	1

4. Label Encoding

Label Encoding asigna un número entero a cada categoría. Por ejemplo, una variable Color podría codificarse como Azul=0, Rojo=1 y Verde=2. Es sencillo y reduce el número de columnas, pero hay que tener cuidado: para variables nominales sin orden, algunos algoritmos pueden interpretar artificialmente que 2 es mayor que 1 y que 1 es mayor que 0.

Color	Código
Azul	0
Rojo	1
Verde	2

5. Diferencias entre las técnicas

Técnica	Resultado	Ventaja	Precaución
KNN	Clasifica/regresa usando vecinos	Simple e intuitivo	La distancia y la escala de las variables importan
One-Hot Encoding	Una columna 0/1 por categoría	No impone orden entre categorías	Puede aumentar mucho el número de columnas
Label Encoding	Un número por categoría	Simple y compacto	Puede introducir un orden artificial en variables nominales

6. Cómo realizarlo en Excel paso a paso

6.1 Preparar los datos

Crea una tabla con datos de ejemplo. Por ejemplo: Edad, Color y Clase. Puedes usar Edad como variable numérica, Color como variable categórica y Clase como resultado.

Edad	Color	Clase	Fila
20	Rojo	A	1
22	Azul	B	2
21	Rojo	A	3
25	Verde	B	4
24	Azul	B	5
19	Rojo	A	6

6.2 One-Hot Encoding en Excel

1. Coloca las categorías como encabezados de nuevas columnas. Por ejemplo: Color_Rojo, Color_Azul y Color_Verde.

2. En la primera fila de datos, escribe una fórmula para comprobar si el valor coincide con el encabezado.

3. Para Color_Rojo, si el color está en B2, puedes usar: =SI($B2=Rojo";1;0)

4. Para Color_Azul: =SI($B2=.ªzul";1;0)

5. Para Color_Verde: =SI($B2="Verde";1;0)

6. Arrastra las fórmulas hacia abajo para todas las filas.

7. El resultado será una matriz de 0 y 1. Esta es la representación One-Hot.

6.3 Label Encoding en Excel

Una forma sencilla es crear una tabla de equivalencias. Por ejemplo: Azul=0, Rojo=1 y Verde=2.

1. En otra zona de la hoja crea una tabla con las categorías y sus códigos.

2. Si tienes Excel moderno, usa BUSCARX para recuperar el código correspondiente.

3. Ejemplo: =BUSCARX(B2;$H$2:$H$4;$I$2:$I$4)

4. Copia la fórmula hacia abajo.

5. Revisa que todas las categorías hayan recibido un código.

6.4 KNN en Excel paso a paso

1. Convierte primero las variables categóricas a números mediante One-Hot Encoding o Label Encoding, según el caso.

2. Si las variables numéricas tienen escalas muy diferentes, considera normalizarlas o estandarizarlas.

3. Define el nuevo punto que deseas clasificar.

4. Calcula la distancia euclidiana entre el nuevo punto y cada observación conocida. Por ejemplo, para dos variables: =RAIZ((A2-$J$2)^2+(B2-$K$2)^2)

5. Copia la fórmula para todas las observaciones.

6. Ordena las distancias de menor a mayor.

7. Selecciona los K vecinos más cercanos. Por ejemplo, si K=3, toma las tres distancias menores.

8. Observa las clases de esos vecinos y asigna al nuevo punto la clase que aparezca con mayor frecuencia.

7. Programación en Python

En Python se pueden utilizar pandas y scikit-learn. OneHotEncoder está disponible en sklearn.preprocessing y KNeighborsClassifier en sklearn.neighbors.

7.1 Instalación

En la terminal o en Google Colab:

pip install pandas scikit-learn

7.2 One-Hot Encoding con Python

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

df = pd.DataFrame({
.Edad": [20, 22, 21, 25, 24, 19],
Çolor": [Rojo", .Azul", Rojo", "Verde", .Azul", Rojo"]
})

encoder = OneHotEncoder(sparse_output=False)
resultado = encoder.fit_transform(df[[Çolor"]])

columnas = encoder.get_feature_names_out([Çolor"])
df_onehot = pd.DataFrame(resultado, columns=columnas)

print(df_onehot)
```

7.3 Label Encoding con Python

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df[Çolor_Label"] = le.fit_transform(df[Çolor"])

print(df)
```

LabelEncoder asigna códigos según las clases que encuentra. Para variables de entrada categóricas, en proyectos reales también conviene evaluar OrdinalEncoder o OneHotEncoder según la naturaleza de la variable y el modelo.

7.4 KNN completo con One-Hot Encoding

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

df = pd.DataFrame({
.Edad": [20, 22, 21, 25, 24, 19, 30, 28, 27, 23],
Çolor": [Rojo", .Azul", Rojo", "Verde", .Azul",
Rojo", "Verde", .Azul", "Verde", Rojo"],
Çlase": [.A", "B", .A", "B", "B", .A", "B", "B", "B", .A"]
})

X = df[[.Edad", Çolor"]]
y = df[Çlase"]

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.3, random_state=42, stratify=y
)

preprocesamiento = ColumnTransformer([
("num", StandardScaler(), [.Edad"]),
(çat", OneHotEncoder(handle_unknown=ïgnore"), [Çolor"])
])

modelo = Pipeline([
("preprocesamiento", preprocesamiento),
("knn", KNeighborsClassifier(n_neighbors=3))
])

modelo.fit(X_train, y_train)

predicciones = modelo.predict(X_test)
```

Universidad Autónoma de TAMAULIPAS
Facultad de Ingeniería Tampico

print("Predicciones:", predicciones)
print(.ExactitudE:", accuracy_score(y_test, predicciones))

8. Ejemplo de predicción

Después de entrenar el modelo, se puede clasificar un nuevo registro. Por ejemplo, una persona de 23 años cuyo color es Azul:

nuevo = pd.DataFrame({
."Edad": [23],
Çolor": [."Azul"]
})

prediccion = modelo.predict(nuevo)
print(Çlase predicha:", prediccion[0])

9. ¿Qué técnica conviene utilizar?

Para KNN, generalmente es importante representar correctamente las variables categóricas porque el algoritmo depende de distancias. Si una variable nominal no tiene un orden natural, One-Hot Encoding suele ser una opción más segura que asignarle números arbitrarios. También es importante escalar las variables numéricas para evitar que una variable con valores grandes domine el cálculo de distancia.

10. Conclusión

KNN es un algoritmo sencillo de aprendizaje automático que clasifica observaciones mediante la proximidad de sus vecinos. One-Hot Encoding transforma categorías en columnas binarias y evita imponer relaciones numéricas artificiales entre categorías. Label Encoding transforma cada categoría en un entero y es compacto, aunque debe utilizarse con precaución cuando las categorías no poseen un orden natural. Excel permite practicar estas transformaciones mediante fórmulas, mientras que Python y scikit-learn permiten automatizar el preprocesamiento y construir modelos KNN de manera reproducible.

11. Referencias

- Scikit-learn. (2026). Nearest Neighbors. Documentación oficial de scikit-learn.
- Scikit-learn. (2026). OneHotEncoder. Documentación oficial de scikit-learn.
- Scikit-learn. (2026). KNeighborsClassifier. Documentación oficial de scikit-learn.
- Microsoft Support. Text Import Wizard. Documentación de Microsoft Excel.

Udacity. (2025). One-Hot Encoding Explained: A Key Step in Data Preprocessin

DATA LEAK

Carrera: Ingeniería en ciencia de datos he inteligencia artificial

Equipo: Alt F4

Fecha:10/09/26

¿Qué son los Data Leaks?

FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109
70 Aniversario Facultad de Ingeniería Tampico 1956-2026

Los **Data Leaks o fugas de datos** son situaciones en las que información confidencial o sensible queda expuesta, filtrada o es accedida por personas que no tienen autorización. Estas filtraciones pueden producirse de manera **accidental o intencional** y representan un riesgo importante para la seguridad de las personas y las empresas.

Una fuga de datos puede ocurrir por diferentes razones. Entre ellas se encuentran las **fallas en los sistemas informáticos, vulnerabilidades de seguridad, ataques de hackers, malware y errores humanos**. Por ejemplo, un empleado puede enviar accidentalmente un correo con información privada a la persona equivocada o una empresa puede dejar una base de datos accesible en Internet sin la protección adecuada.

La información que puede quedar expuesta incluye **datos personales, contraseñas, información financiera, datos de clientes, documentos internos, secretos comerciales y estrategias de una empresa**. Por esta razón, un Data Leak puede afectar tanto a organizaciones como a usuarios individuales.

¿Cómo se produce un Data Leak?

Existen diferentes formas en las que puede producirse una fuga de datos. Una de las principales son las **brechas de seguridad**, en las que un atacante logra superar las medidas de protección de un sistema y obtiene acceso a información privada.

Otra causa es el **malware**, como virus, troyanos y otros programas maliciosos que pueden infectar dispositivos y robar información sin que el usuario lo note. Los datos obtenidos pueden posteriormente ser utilizados de manera ilegal o publicados.

También es muy importante el **error humano**. Acciones aparentemente simples, como enviar un archivo confidencial a un destinatario equivocado, utilizar contraseñas inseguras o dejar información almacenada en un lugar público, pueden provocar una filtración.

Impacto de un Data Leak

Las consecuencias de una fuga de datos pueden ser graves. Para las empresas, uno de los principales problemas es el **daño a su reputación**, ya que los clientes pueden perder la confianza al saber que su información no fue protegida correctamente.

También pueden producirse **pérdidas económicas**. Una empresa puede tener que pagar por investigaciones, recuperación de información, notificación a los afectados y medidas para solucionar el problema. Además, puede perder clientes y oportunidades de negocio.

Otro riesgo importante es la **exposición de información sensible**. Los datos filtrados pueden ser utilizados por delincuentes para realizar fraudes, robo de identidad u otras actividades maliciosas. En el caso de información empresarial, los competidores podrían aprovechar secretos comerciales o estrategias internas.

Para las personas, una filtración puede representar una **violación de la privacidad**. Si se exponen datos personales, estos pueden utilizarse para suplantar la identidad de una persona, cometer fraudes o realizar otros ataques.

Prevención de los Data Leaks

Para reducir el riesgo de sufrir una fuga de datos, las empresas deben implementar diferentes medidas de seguridad. Una de las más importantes es la **capacitación del personal**, ya que los trabajadores deben conocer los riesgos de compartir información y aprender a utilizar correctamente correos electrónicos, contraseñas y sistemas de almacenamiento.

También es necesario establecer **políticas de seguridad de datos**, controlar quién puede acceder a determinada información y utilizar mecanismos como el cifrado para proteger los datos.

Las herramientas de **Data Loss Prevention (DLP)** ayudan a identificar, supervisar y controlar el movimiento de información confidencial dentro y fuera de una organización. Estas herramientas pueden detectar actividades sospechosas y evitar que determinados datos sean enviados o compartidos de manera incorrecta.

Las **copias de seguridad** también son importantes porque permiten recuperar información en caso de que esta sea perdida, dañada o afectada durante un incidente de seguridad. Además, es necesario mantener los programas y sistemas **actualizados**, ya que las actualizaciones suelen corregir vulnerabilidades que podrían ser aprovechadas por atacantes.

Por último, las empresas deben realizar **auditorías de seguridad, análisis de vulnerabilidades y pruebas de penetración** para encontrar posibles debilidades antes de que sean aprovechadas por los ciberdelincuentes.

Conclusión

En conclusión, un **Data Leak es una exposición no autorizada de información que puede ocurrir por ataques cibernéticos, malware, fallas de seguridad o errores humanos**. Sus consecuencias pueden afectar la privacidad de las personas y provocar pérdidas económicas, robo de identidad, exposición de información confidencial y daños a la reputación de las empresas.

Por ello, es fundamental aplicar medidas de prevención como la **capacitación del personal, políticas de seguridad, control de accesos, herramientas DLP, copias de seguridad, actualizaciones y auditorías periódicas**. La protección de los datos debe ser una responsabilidad constante, ya que prevenir una filtración es mucho más seguro que intentar solucionar sus consecuencias después de que ocurra.

Referencias:

¿Qué son los Data Leaks? | Trend Micro (ES)

¿Qué es un Data Leak y cómo evitarlo?

Data Leakage: riesgos, causas y prevención | Wiz

¿Qué es un Data Leak y por qué debes preocuparte por él? - Herschel González

Errores Mae, Mse, Rmse, Mape

Carrera: Ingeniería en ciencia de datos he inteligencia artificial

Equipo: Alt F4

Fecha:10/09/26

Error Absoluto Medio (MAE)

¿Qué es el Error Absoluto Medio?

El **Error Absoluto Medio (MAE)**, conocido en inglés como **Mean Absolute Error**, es una medida que permite conocer qué tan diferentes son unos valores predichos respecto a los valores reales.

Para calcularlo, se obtiene la diferencia entre el valor real y el valor predicho, se utiliza el valor absoluto para eliminar los signos negativos y finalmente se calcula el promedio de todos los errores.

La fórmula es:

$$\mathbf{MAE} = \sum \mathbf{|Valor\ real - Valor\ predicho|\ /\ n}$$

Donde:

- **Valor real:** es el valor que realmente se obtuvo.
- **Valor predicho:** es el valor que se había estimado.
- **| |:** representa el valor absoluto.
- **n:** es la cantidad de datos utilizados.

Un MAE pequeño significa que, en promedio, existe poca diferencia entre los valores reales y los valores predichos.

Ejemplo 1: Puntos de experiencia en un videojuego

Supongamos que un jugador realiza diferentes misiones y se quiere comparar la cantidad de experiencia que realmente obtuvo con una cantidad que había sido estimada.

Misión	Experiencia real	Experiencia predicha	Error absoluto
1	500	450	50
2	800	850	50
3	1,000	950	50
4	1,200	1,250	50

Calculamos el MAE:

MAE = (50 + 50 + 50 + 50) / 4

MAE = 200 / 4

MAE = 50 XP

Interpretación

El error absoluto medio es de **50 puntos de experiencia**. Esto significa que, en promedio, la diferencia entre la experiencia real y la experiencia predicha es de 50 XP.

Conclusión

El **Error Absoluto Medio (MAE)** es una medida utilizada para conocer la diferencia promedio entre los valores reales y los valores predichos.

Su cálculo consiste en obtener el error de cada dato, convertirlo a un valor absoluto, sumar todos los errores y dividir el resultado entre la cantidad de datos.

Los ejemplos con videojuegos permiten observar de una manera sencilla cómo se puede calcular el MAE utilizando situaciones conocidas, como la experiencia obtenida, las monedas, el daño, el tiempo de las misiones y los niveles alcanzados.

En general, **un MAE menor representa una menor diferencia promedio entre los valores reales y los predichos**.

Ahora sí, los videojuegos están usados **solamente como contexto de los ejemplos**, no como una aplicación específica del MAE.

Error Cuadrático Medio (MSE)

¿Qué es el MSE?

El **Error Cuadrático Medio (MSE)** es una medida estadística utilizada para saber **qué tan buenas son las predicciones de un modelo**.

Compara los valores que el modelo **predice** con los valores que realmente **ocurren**. Después, eleva las diferencias al cuadrado y calcula su promedio.

Un **MSE bajo** significa que las predicciones están cerca de los valores reales.
Un **MSE alto** significa que existen errores grandes en las predicciones.

Ejemplo en videojuegos

Imaginemos un modelo que intenta predecir el **daño que hará un personaje en un videojuego**.

- Daño real: 100
- Daño predicho: 90
- Error: $100 - 90 = 10$
- Error cuadrado: $10^2 = 100$

Si hacemos esto con muchas partidas y calculamos el promedio, obtenemos el MSE.

Fórmula del MSE

La fórmula es:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(\ Y_i - \widehat{Y}_i \ )^{\ 2}$$

Donde:

- Y_i= valor real.
- $\widehat{Y}_i$= valor predicho por el modelo.
- n= cantidad de observaciones.

Ejemplo

Un modelo predice los puntos que obtendrá un jugador:

Partida	Puntos reales	Puntos predichos	Error²
1	100	90	100
2	200	190	100
3	300	250	2500

Entonces:

$$MSE = \frac{100 + 100 + 2500}{3} MSE = 900$$

El MSE es **900**.

Esto muestra que la tercera predicción tuvo un error mucho mayor y afectó bastante el resultado.

Conclusión

El **Error Cuadrático Medio (MSE)** es una métrica que permite medir **qué tan alejadas están las predicciones de un modelo respecto a los valores reales**. Para calcularlo, se obtiene la diferencia entre el valor real y el predicho, se eleva al cuadrado y finalmente se calcula el promedio.

Una de sus características principales es que **castiga más los errores grandes**, por lo que resulta útil cuando estos errores son importantes. En general, **un MSE más bajo representa un modelo con mejores predicciones**.

En videojuegos puede utilizarse, por ejemplo, para evaluar modelos que predicen **daño, puntuaciones, tiempos de juego, rendimiento de jugadores o resultados de partidas**.

¿Qué es el RMSE?

El **RMSE (Root Mean Squared Error)** o **raíz del error cuadrático medio** es una medida que sirve para saber **qué tan equivocadas están, en promedio, las predicciones de un modelo**.

Compara los valores que el modelo **predice** con los valores que realmente **ocurren**.

Una característica importante es que **da mayor importancia a los errores grandes**, porque primero eleva las diferencias al cuadrado.

Fórmula

$$RMSE = \sqrt{\frac{\sum(\ P_i - O_i\ )^{\ 2}}{n}}$$

Donde:

- P_i= valor predicho por el modelo.
- O_i= valor observado o real.
- n= cantidad de datos.

¿Cómo se calcula?

1. Se resta el valor real al valor predicho.
2. Se eleva el resultado al cuadrado.
3. Se suman todos los errores cuadrados.
4. Se divide entre el número de datos para obtener el **MSE**.
5. Se obtiene la raíz cuadrada del MSE para obtener el **RMSE**.

En pocas palabras: el RMSE indica aproximadamente **cuánto se equivoca un modelo al hacer sus predicciones**.

Ejemplo 1: Daño de un personaje

Supongamos que un modelo intenta predecir cuánto daño hará un personaje en un videojuego.

Daño real	Daño predicho
100	95
150	140
200	180

Los errores serían:

- $95 - 100 = -5$
- $140 - 150 = -10$
- $180 - 200 = -20$

Al elevarlos al cuadrado:

- $(-5)^2 = 25$
- $(-10)^2 = 100$
- $(-20)^2 = 400$

Entonces:

$$RMSE = \sqrt{\frac{25 + 100 + 400}{3}} RMSE = \sqrt{175} RMSE \approx 13{,}23$$

Esto significa que el modelo tiene un **error aproximado de 13.23 puntos de daño** en sus predicciones.

Conclusión

El **RMSE es una métrica utilizada para medir el error de las predicciones de un modelo**. Se obtiene calculando la diferencia entre los valores reales y predichos, elevando esas diferencias al cuadrado, obteniendo su promedio y finalmente sacando la raíz cuadrada.

Su principal característica es que **penaliza más los errores grandes**, por lo que es útil cuando queremos evitar predicciones muy alejadas de la realidad.

Un **RMSE menor generalmente significa que el modelo realiza mejores predicciones**, aunque siempre debe analizarse de acuerdo con el contexto y las unidades de los datos.

En videojuegos, por ejemplo, puede utilizarse para evaluar modelos que predicen **daño, puntuaciones, tiempos de partida, niveles de jugadores, precios de objetos o cualquier otro valor numérico**.

MAPE (Error Porcentual Absoluto Medio)

El **MAPE (Mean Absolute Percentage Error)** es una métrica que sirve para medir **qué tan diferentes son las predicciones de los valores reales**, pero expresando el error como **porcentaje**.

Dicho de forma sencilla, responde a la pregunta:

¿En qué porcentaje se equivoca, en promedio, una predicción respecto al valor real?

Fórmula

$$MAPE = \frac{1}{n} \sum_{i=1}^{n} \left| \frac{y_i - \widehat{y_i}}{y_i} \right| \times 100$$

Donde:

- y_i= valor real.
- $\widehat{y_i}$= valor predicho.
- n= cantidad de datos.

Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

- El resultado se expresa en **porcentaje**.

¿Cómo se calcula?

1. Se resta el valor real y el predicho.
2. Se obtiene el **error absoluto**.
3. Se divide entre el valor real.
4. Se multiplica por 100 para obtener el porcentaje.
5. Se promedian todos los porcentajes.

Ejemplos con videojuegos

Ejemplo 1: predicción de jugadores

Un juego tuvo realmente **1,000 jugadores** en un día, pero el modelo predijo **900**.

$$\frac{|\,1000 - 900\,|}{1000} \times 100 = 10\,\%$$

El error de la predicción fue del **10 %**.

Ejemplo 2: daño de un personaje

Un modelo predice que un personaje hará **500 de daño**, pero realmente hace **550**.

$$\frac{|\,550 - 500\,|}{550} \times 100 \approx 9{,}09\,\%$$

El error porcentual es aproximadamente **9.09 %**.

Ejemplo 3: objetos obtenidos

Un jugador realmente consigue **50 objetos** durante una partida y el modelo predice **40**.

$$\frac{|\,50 - 40\,|}{50} \times 100 = 20\,\%$$

El error es del **20 %**.

¿Cómo interpretar el MAPE?

- **MAPE = 5 %:** las predicciones se equivocan aproximadamente un 5 % en promedio.
- **MAPE = 10 %:** el error promedio es de aproximadamente 10 %.
- **MAPE = 20 %:** las predicciones tienen un error promedio considerablemente mayor.

En general, **mientras menor sea el MAPE, más cercanas están las predicciones a los valores reales**. Sin embargo, no existe un porcentaje que sea automáticamente "bueno"para todos los casos.

Ventajas

- Es fácil de entender porque se expresa en **porcentaje**.
- Permite comparar errores aunque los valores tengan diferentes escalas.
- Ayuda a evaluar modelos de predicción.

VERDAD, BELLEZA, PROBIDAD
FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109
70 Aniversario Facultad de Ingeniería Tampico 1956-2026

Limitaciones

El principal problema del MAPE aparece cuando el **valor real es 0 o está muy cerca de 0**. En esos casos puede producir valores extremadamente grandes o incluso no poder calcularse.

Por ejemplo, si un videojuego tiene realmente **0 jugadores** en una determinada medición, no se puede dividir el error entre 0.

Por eso, en algunos casos se utilizan otras métricas como **MAE, RMSE, sMAPE o WMAPE**.

En resumen

MAPE mide el error promedio de las predicciones en forma de porcentaje. Permite saber de manera sencilla qué tan lejos están las predicciones de los valores reales. Un MAPE bajo indica predicciones más precisas, aunque debe tenerse cuidado cuando los valores reales son cero o muy pequeños.

UNIVERSIDAD AUTÓNOMA DE TAMAULIPAS
VERDAD, BELLEZA, PROBIDAD

Investigacion: metricas obtenibles de una matriz de confusion

Una matriz de confusión es una de las herramientas más útiles e intuitivas en el campo de la Inteligencia Artificial y la Ciencia de Datos para evaluar qué tan bien se está comportando un modelo de clasificación (por ejemplo, cuando usas algoritmos como k-NN o regresión logística en Scikit-learn).

En lugar de decirte simplemente .[el] modelo acertó el 90 % de las veces", la matriz desglosa dónde se equivocó y cómo se equivocó.

La Estructura (2x2 para clasificación binaria)

Imagina que entrenaste un modelo para predecir si un correo es Spam (Positivo) o No Spam (Negativo). Cuando comparas lo que el modelo predijo contra lo que realmente era el correo, los resultados caen en una de cuatro categorías, formando una cuadrícula de 2x2:

	Predicción: Positivo (Es Spam)	**Predicción: Negativo** (No Spam)
Realidad: Positivo (Era Spam)	**Verdadero Positivo (TP):** El modelo acertó.	**Falso Negativo (FN) - Error Tipo II:** Dijo que no era spam, pero sí lo era.
Realidad: Negativo (No era Spam)	**Falso Positivo (FP) - Error Tipo I:** Dijo que era spam, pero era un correo normal.	**Verdadero Negativo (TN):** El modelo acertó.

- **TP (True Positive) y TN (True Negative):** Son las predicciones correctas. Queremos que estos números sean lo más altos posible.
- **FP (False Positive) y FN (False Negative):** Son los errores. Dependiendo del problema, un error es más grave que otro. (En un filtro de spam, un FP significa mandar un correo importante de tu profesora a la papelera; un FN es simplemente recibir un correo basura en tu bandeja).

¿Por qué es mejor que la simple .[Ex]actitud"(Accuracy)?

Si tienes un conjunto de datos desbalanceado (ej. 99 correos normales y 1 spam), un modelo "tonto"que siempre diga "No es spam"tendrá un 99 % de exactitud. Parece excelente, pero en realidad es inútil porque falló en detectar el único correo de spam (tuvo un FN). La matriz de confusión pone esto en evidencia inmediatamente y nos permite calcular métricas más robustas:

- **Precisión (Precision):** De todos los que el modelo dijo que eran Positivos, ¿cuántos realmente lo eran? Es muy útil cuando los Falsos Positivos son costosos.

$$\text{Precisión} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **Sensibilidad o Exhaustividad (Recall / True Positive Rate):** De todos los Positivos reales que había en los datos, ¿cuántos logró encontrar el modelo? Es vital cuando los Falsos Negativos son peligrosos (ej. detectar una enfermedad).

$$\text{Sensibilidad} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **F1-Score:** Es una media armónica entre Precisión y Sensibilidad. Es la mejor métrica si tienes clases

Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

muy desbalanceadas.

$$\text{F1-Score} = 2 \cdot \frac{\text{Precisión} \cdot \text{Sensibilidad}}{\text{Precisión} + \text{Sensibilidad}}$$

¿Cómo se ve en código?

Si estás trabajando en un entorno como PyCharm y tienes tus predicciones listas (por ejemplo, en un proyecto de fundamentos de ML), Scikit-learn lo hace en dos líneas:

```
from sklearn.metrics import confusion_matrix

# y_true son los datos reales, y_pred son las predicciones de tu modelo
matriz = confusion_matrix(y_true, y_pred)
print(matriz)
```

Conclusión General

La matriz de confusión transforma una evaluación numérica simple en un mapa de diagnóstico operacional. La clave para evaluar un modelo exitosamente no radica en obtener un porcentaje alto en todas las métricas, sino en alinear la métrica prioritaria con la consecuencia de equivocarse en el mundo real: reducir Falsos Positivos cuando la falsa alarma sea costosa, o reducir Falsos Negativos cuando la omisión sea riesgosa.

Síntesis de Métricas y su Criterio de Selección

- **Exactitud (Accuracy):**
  - *Uso:* Evalúa el rendimiento general del modelo cuando las clases están bien balanceadas.
  - *Limitación:* Es engañosa en conjuntos de datos desbalanceados, ya que oculta fallos severos en la clase minoritaria.
- **Precisión (Precision):**
  - *Uso:* Prioritaria cuando los Falsos Positivos son costosos o dañinos.
  - *Objetivo:* Minimizar falsas alarmas (por ejemplo, evitar marcar correos legítimos como spam o enviar notificaciones erróneas a clientes).
- **Sensibilidad / Exhaustividad (Recall):**
  - *Uso:* Prioritaria cuando los Falsos Negativos son críticos o peligrosos.
  - *Objetivo:* Capturar la mayor cantidad posible de casos positivos reales (por ejemplo, en el diagnóstico de enfermedades graves o detección de fraudes transaccionales).
- **Puntuación F1 (F1-Score):**
  - *Uso:* Funciona como el balance óptimo (media armónica) entre Precisión y Sensibilidad.
  - *Objetivo:* Es el indicador de referencia recomendado para medir el rendimiento real en problemas con clases desbalanceadas.

FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109

70 Aniversario Facultad de Ingeniería Tampico 1956-2026

Referencias:

Mean absolute percentage error - Wikipedia

MAPE: qué es, fórmula y cómo mejorar tu forecast

MAPE (Error Porcentual Absoluto Medio) — Métrica de Evaluación en Regresión para Interpretar la Precisión Predictiva a Partir del Error Porcentual Medio Respecto al Valor Real - Zero Math AI

Error Porcentual Absoluto Medio (MAPE) en un Pronóstico de Demanda

Error cuadratico medio MSE Error cuadratico medio una inmersion mas profunda en la varianza y la precision del modelo - FasterCapital

Error cuadrático medio - Wikipedia, la enciclopedia libre

Error cuadrado medio (MSE) - Dataconomy ES

¿Qué es el error cuadrado de raíz (RMSE)? - Dataconomy ES

Raíz del error cuadrático medio - Wikipedia, la enciclopedia libre

Explicación del RMSE: Guía para la precisión de la predicción de regresión | DataCamp

Error absoluto medio explicado: Medición de la precisión del modelo | DataCamp

Error absoluto medio (MAE) - Dataconomy ES

Qué es: Error absoluto medio (MAE): APRENDA ESTADÍSTICAS FÁCILMENTE

- Matriz de confusión - Wikipedia, la enciclopedia libre
- ¿Qué es una matriz de confusión? - Dataconomy ES
- Guía interactiva de la matriz de confusión y sus métricas - Dataconomy ES
- Precisión y Sensibilidad explicadas - Dataconomy ES
- Explicación de la matriz de confusión: Guía para evaluar modelos de clasificación | DataCamp
- Métricas de clasificación explicadas: De la matriz de confusión al F1-Score | DataCamp
- Precisión, Sensibilidad (Recall) y F1-Score: Entendiendo el balance de las métricas en clasificación — FasterCapital
- Matriz de confusión y métricas de evaluación (Precisión, Recall, F1-score, Accuracy) - Zero Math AI
- Matriz de confusión
- Matriz de confusión — Evaluando el rendimiento de un modelo de clasificación
- Matriz de confusión en Machine Learning: Guía completa para interpretar resultados
- Qué es: Matriz de confusión en clasificación: APRENDA ESTADÍSTICAS FÁCILMENTE

Documentación de los códigos hechos en clase

Equipo ALTF4
MARTINEZ GIANNCARLO
BANDA HERNANDEZ JUAN CARLOS
MENDEZ GOMEZ JUAN CARLOS

Documentación del código de regresión multivariada

Empecemos definiendo la regresión lineal multivariada. La regresión lineal multivariada es usada para predecir el valor de una variable dependiente (continua) a partir de múltiples variables independientes, calcula una combinación lineal de las variables de entrada:

$$y = b_0 + b_1x_1 + b_2x_2 + \cdots + b_nx_n$$

Para encontrar los coeficientes óptimos (b), este código aplica un método algebraico directo conocido como la Ecuación Normal de Mínimos Cuadrados Ordinarios:

$$b = (X^T \cdot X)^{-1} \cdot X^T \cdot Y$$

Para hacer una demostración de la regresión multivariada, el script analizara los datos del archivo `'Advertising.csv'`, en el cual se busca predecir las ventas (`'Sales'`) basándose en la inversión en distintos medios de publicidad (`'TV'`, `'Radio'`, `'Newspaper'`). La variable `'auxiliar'` actúa como el término de sesgo o intercepto (b_0).

```
import pandas as pd
import numpy as np

datos = pd.read_csv('advertising.csv')
X = datos[['auxiliar','TV', 'Radio', 'Newspaper']].to_numpy()
Y = datos['Sales'].to_numpy()

b = X.T.dot(X)
b = np.linalg.inv(b)
b = b.dot(X.T)
b = b.dot(Y)
print(b)

Y_pred = X.dot(b)
error = 0
for i in range(Y_pred.shape[0]):
    error += (Y[i] - Y_pred[i])** 2
    print( str(Y[i]) + "\t\t" + str(round(Y_pred[i],1)))
print()
error /= Y_pred.shape[0]
print("RMSE: " + str(error**(1/2)))
```

Desglose del código

Usamos las librerías `pandas` y `numpy`. Pandas se encarga de la lectura del archivo CSV, mientras que NumPy es utilizado para transformar los datos a matrices y realizar operaciones de álgebra lineal avanzado.

Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

Definición de los Datos

```
datos = pd.read_csv('advertising.csv')
X = datos[['auxiliar','TV', 'Radio', 'Newspaper']].to_numpy()
Y = datos['Sales'].to_numpy()
```

- **X:** Matriz bidimensional de características que representa la inversión en publicidad y el término 'auxiliar' (usado para el sesgo o intercepto, lleno de 1s).
- **Y:** Vector unidimensional con los datos reales de ventas (Sales), que funcionará como variable objetivo a predecir.

Cálculo de los Coeficientes (Entrenamiento del Modelo)

```
b = X.T.dot(X)
b = np.linalg.inv(b)
b = b.dot(X.T)
b = b.dot(Y)
```

En este bloque, el código calcula los coeficientes (pesos) óptimos empleando álgebra de matrices.

- `X.T.dot(X)`: Transpone la matriz X y la multiplica por sí misma.
- `np.linalg.inv(b)`: Calcula la inversa de la matriz resultante.
- `b.dot(X.T)` y `b.dot(Y)`: Multiplica la inversa por la matriz transpuesta y finalmente por el vector Y para encontrar el vector de coeficientes b.

Predicciones y Evaluación de Errores

```
Y_pred = X.dot(b)
```

Se calcula la matriz de predicciones (`Y_pred`) multiplicando las características originales X por los coeficientes encontrados b.
El bucle `for` compara iterativamente el valor real de ventas con la predicción del modelo y muestra los resultados, de ahí como último paso se calcula el Error Cuadrático Medio (RMSE) calculando el promedio de la diferencia al cuadrado entre lo real y lo predicho, aplicándole al final una raíz cuadrada.

Resultados entregados por el código

El código imprimirá primero el arreglo de los coeficientes b (el intercepto y los pesos correspondientes para TV, Radio y Newspaper). Luego, enlista el desglose completo del valor real de venta esperado lado a lado de la predicción calculada redondeada a un decimal. Para terminar, muestra el valor RMSE (Root Mean Square Error), que equivale a 1.66, esto significa que en promedio, las predicciones del modelo de regresión multivariada se desvían en 1.66 unidades del valor real de las ventas, aunque esto necesita contexto, ya que podrían ser pesos, dólares, euros, manzanas, peras, etc.

Documentación del código de regresión logística

Primero abramos definiendo la regresión logística. La regresión logística es usada para resolver problemas de clasificación cuando a la variable es del tipo binario, esta aplica una función matemática llamada sigmoide la cual primero calcula una combinación lineal tradicional de las variables de entrada:

$$z = b_0 + b_1x_1 + b_2x_2 + \cdots + b_nx_n$$

FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109
70 Aniversario Facultad de Ingeniería Tampico 1956-2026

El valor z se introduce dentro de la función sigmoide para que arroje un valor entre 0 y 1 para clasificar cada caso. Este resultado se puede interpretar directamente como la probabilidad de que ocurra un evento u otro.

Para hacer una demonstración de la regresión logística hicimos un script simple en el que en un escenario ficticio se predice si un alumno aprueba o no dependiendo de las horas estudiadas. x significa las horas estudiadas mientras que y es el caso binario de si aprobó o no, 0 significa que no aprobó y 1 significa que si aprobó.

```
from sklearn.linear_model import LogisticRegression

X=[[1],[2],[3],[4],[5],[6],[7],[8]]
y= [0,0,0,0,1,1,1,1]

modelo = LogisticRegression()
modelo.fit(X,y)

print("Intercepto: ",modelo.intercept_)
print("Coeficiente: ",modelo.coef_)

n1 = [[4.5]]
p1= modelo.predict(n1)
pb1= modelo.predict_proba(n1)

print("Prediccion", p1)
print("Probabilidad", pb1)
```

Desglose del código

Usamos el módulo `sklearn.linear_model.LogisticRegression`. Esta es la clase principal de la librería `scikit-learn` encargada de implementar el algoritmo de regresión logística. Se encarga de ajustar una curva sigmoide a los datos de entrenamiento para calcular probabilidades de pertenencia a una clase.

Definición de los Datos

```
X = [[1], [2], [3], [4], [5], [6], [7], [8]]
y = [0, 0, 0, 0, 1, 1, 1, 1]
```

- **X:** Matriz bidimensional donde cada fila representa las horas de estudio de un alumno.
- **y:** Lista unidimensional que indica el resultado de cada alumno (0 = No aprobó, 1 = Los primeros 4 alumnos estudiaron poco y reprobaron, mientras que del 5 al 8 estudiaron más y aprobaron lo que funcionara como base para la predicción).

Creación y Entrenamiento del Modelo

```
modelo = LogisticRegression()
modelo.fit(X, y)
```

- `LogisticRegression()`: Instancia (crea) el objeto del modelo utilizando los parámetros por defecto de scikit-learn.
- `modelo.fit(X, y)`: Entrena el modelo. En este paso, el algoritmo analiza los datos de X y y para encontrar los parámetros óptimos que separan de mejor manera ambas clases mediante la función logística.

Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

Extracción de Parámetros del Modelo

```
print("Intercepto: ", modelo.intercept_)
print("Coeficiente: ", modelo.coef_)
```

- `modelo.intercept_` (b_0): Es el sesgo de la función lineal antes de pasar por la función sigmoide.
- `modelo.coef_` (b_1): Es el coeficiente asignado a la variable X. Indica qué tanto impacta una hora extra de estudio en la probabilidad de aprobar.

Predicciones y Estimación de Probabilidades

El código evalúa cuatro nuevos casos de estudio (4.5, 5, 6 y 7 horas) utilizando dos funciones clave:

- `modelo.predict(n)`: Devuelve la clase estimada de forma categórica (en este caso, 0 o 1) basándose en un umbral de decisión predeterminado (generalmente 0.5).
- `modelo.predict_proba(n)`: Devuelve un arreglo con las probabilidades para cada clase PClase 0, PClase 1. La suma de ambas probabilidades siempre es 1.0.

Funciones Principales Empleadas de Scikit-Learn

- `.fit(X, y)`: Ajusta el modelo a los datos de entrenamiento.
- `.predict(X)`: Predice la etiqueta de clase para las muestras ingresadas.
- `.predict_proba(X)`: Calcula las probabilidades estimadas para cada clase.

Resultados entregados por el código

Como podemos apreciar el que estudio 4.5 horas por una probabilidad muy pequeña no aprobo el examen los que estudiaron 5 horas en adelante si lo hicieron con una tasa de éxito diferente. El que mas probabilidades tuvo de aprobar fue el que estudio 7 horas por lo que esta sería la estrategia que tendrías que tomar si quieres asegurarte de aprobar.

VERDAD, BELLEZA, PROBIDAD

Documentación del código de regresión logística multiclase (One-vs-Rest)

1. Documentación del archivo Excel: Actividad_ML_10_6_2026_completo.xlsx

Tras analizar la estructura del documento Excel, se observa que funciona como el respaldo matemático y de cálculo manual de los modelos de regresión logística que se ejecutan en el código.

Estructura y contenido principal del archivo

- **Mapeo de clases:** El archivo define la equivalencia de 4 clases de vehículos que está intentando predecir el modelo. En las columnas de la derecha se establece la relación: la Clase 1 corresponde a SUV, la Clase 2 a Sedán, la Clase 3 a Deportivo y la Clase 4 a Camioneta.

	Clase 1	Clase 2	clase 3	clase 4
	SUV	Sedan	Deportivo	Camioneta

Figura 1: Mapeo de clases en el archivo Excel de la regresión logística.

- **Parámetros del modelo (coeficientes):** Las columnas principales estructuran la ecuación de la regresión logística. La columna b0 representa el intercepto o sesgo del modelo de cada clase. Las columnas de b1 a b5 representan los coeficientes o pesos asignados a cada uno de los 5 atributos que usa el código: `velocidad_max`, `aceleracion`, `capacidad_carga`, `eficiencia_combustible` y `seguridad`.
- **Cálculos de probabilidad manuales:** Las columnas sin nombre muestran, paso a paso, la aplicación de la función sigmoide: el cálculo de la variable lineal Z, el cálculo exponencial y la probabilidad final.

	A	B	C	D	E	F	G	H
1		b0	b1	b2	b3	b4	b5	
2	clase 1	-3.5067	0.0248	0.0197	0.0078	-0.041	0.0209	
3	clase 2	0.1248	-0.0087	0.0098	-0.0112	0.0116	-0.0103	
4	clase 3	-1.0609	0.0028	-0.0219	-0.0063	0	0.0198	
5	clase 4	-1.0609	0.0028	-0.0219	-0.0063	0	0.0198	
6								

Figura 2: Parámetros del modelo y cálculos de probabilidad en Excel.

Las fórmulas empleadas en el archivo son:

$$Z = b_0 + b_1X_1 + b_2X_2 + b_3X_3 + b_4X_4 + b_5X_5$$

El exponencial se calcula como e^{-Z}, y la probabilidad usando la función sigmoide como:

$$P = \frac{1}{1 + e^{-Z}}$$

Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

Este archivo sirve para auditar los resultados que arroja Python, comprobando que las matemáticas subyacentes de las probabilidades de predicción sean correctas.

2. Documentación del código Python paso a paso

El código implementa una técnica de clasificación multiclase conocida como One-vs-Rest (OvR) o Uno contra Todos, utilizando Regresión Logística. Dado que la regresión logística tradicional es binaria, es decir, solo predice entre 2 resultados, este código entrena varios modelos para manejar múltiples clases.

Paso 1: Importación de librerías

Se utiliza `pandas` para el manejo, lectura y manipulación de los datos en forma de DataFrames o tablas. También se importa `LogisticRegression` desde la librería scikit-learn. Este es el algoritmo matemático que calculará la probabilidad de que un vehículo pertenezca a una clase específica en función de sus características.

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
```

Paso 2: Carga de datos y definición de variables

Se lee el archivo base `Entrenamiento.csv`, que contiene los datos históricos de los vehículos. Se usa la codificación `utf-8-sig` para evitar problemas con caracteres especiales como acentos. Luego, se define la lista `atributos`, que son las variables independientes X que el modelo usará para aprender a diferenciar los autos.

```
df = pd.read_csv("Entrenamiento.csv", encoding="utf-8-sig")

atributos = ["velocidad_max", "aceleracion", "capacidad_carga",
             "eficiencia_combustible", "seguridad"]
```

Paso 3: Identificación de clases y preparación del diccionario de modelos

Se extraen todas las categorías únicas de la columna `Clase`. Se usa `str.strip()` para limpiar cualquier espacio en blanco accidental en el texto y `unique()` para no repetir nombres. Posteriormente, se crean dos diccionarios vacíos: `sub_dataframes`, para guardar tablas de datos transformadas, y `modelos`, para guardar un modelo entrenado por cada clase.

```
clases_unicas = df["Clase"].str.strip().unique()

sub_dataframes, modelos = {}, {}
```

Paso 4: Entrenamiento Uno contra Todos (bucle iterativo)

Aquí radica el núcleo de la estrategia. El código recorre cada clase, por ejemplo, SUV. Primero, copia los datos originales. Luego, realiza una binarización, reemplazando la columna `Clase` actual: si el auto es un SUV, le asigna el valor `si`; si es cualquier otra cosa, como Sedán o Camioneta, le asigna el valor `no`. Después, realiza la separación de variables, donde X almacena los números de los atributos y la variable y se convierte a 1 si es la clase actual y 0 si es el resto. Finalmente, ocurre el entrenamiento, inicializando un modelo con `max_iter=1000`, es decir, hasta 1000 iteraciones para encontrar la fórmula matemática sin fallar. Se guarda este modelo específico en el diccionario y el proceso se repite hasta tener un modelo por cada tipo de vehículo.

```
for clase in clases_unicas:
    sub_df = df.copy()
    sub_df["Clase"] = sub_df["Clase"].apply(
        lambda x: "si" if str(x).strip() == clase else "no")
```

FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109
70 Aniversario Facultad de Ingeniería Tampico 1956-2026

```
    sub_dataframes[clase] = sub_df

    X = sub_df[atributos].values
    y = (sub_df["Clase"] == "si").astype(int).values  # si=1, no=0

    modelo = LogisticRegression(max_iter=1000)
    modelo.fit(X, y)
    modelos[clase] = modelo
```

Paso 5: Creación de datos nuevos para inferencia (predicción)

Se crea un nuevo DataFrame con 4 vehículos desconocidos y sus respectivas características métricas de velocidad, aceleración, carga, entre otras. El objetivo es predecir qué tipo de vehículo es cada uno de ellos.

```
nuevos = pd.DataFrame(
    [
        [95, 90, 15, 35, 65],
        [60, 50, 85, 55, 90],
        [72, 62, 40, 80, 78],
        [50, 40, 95, 45, 82],
    ],
    columns=atributos,
)
```

Paso 6: Cálculo de probabilidades combinadas

Mediante la función `predict_proba`, por cada modelo entrenado, se le pide que evalúe a los 4 nuevos vehículos, lo cual devuelve la probabilidad. Se arma una tabla llamada `probabilidades`. Si hay 4 clases, esta tabla tendrá 4 columnas correspondientes a la probabilidad de ser SUV, probabilidad de ser Sedán, etc. Usando `idxmax(axis=1)`, se busca en cada fila cuál columna tuvo el porcentaje de probabilidad más alto. La clase ganadora se asigna a una nueva columna llamada `Clase_predicha` para esos vehículos nuevos.

```
probabilidades = pd.DataFrame(
    {c: m.predict_proba(nuevos.values)[:, 1] for c, m in modelos.items()}
)
nuevos["Clase_predicha"] = probabilidades.idxmax(axis=1)
```

Paso 7: Despliegue de resultados matemáticos en pantalla

Este bucle final imprime de forma estética los parámetros internos del modelo, que deberían coincidir con los valores b0 a b5 vistos en el Excel. Imprime el intercepto, luego empareja los nombres de los atributos con sus respectivos coeficientes y los imprime. Finalmente, imprime para cada vehículo las probabilidades crudas: $P(no)$, que es la probabilidad de que pertenezca al resto, y $P(si)$, que es la probabilidad de que sea de la clase específica evaluada.

```
for clase, modelo in modelos.items():
    # predict_proba devuelve dos columnas: [P(no), P(si)]
    proba = modelo.predict_proba(nuevos[atributos].values)

    print("=" * 50)
    print(f"SUB-DATAFRAME: {clase} vs. resto")
    print("=" * 50)
    print("Intercepto:", modelo.intercept_[0])
    print("Coeficientes:")
    for nombre, coef in zip(atributos, modelo.coef_[0]):
        print(f"  {nombre}: {coef:.4f}")
    print("Probabilidades por vehículo nuevo:")
```

Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

```
    for i, (p_no, p_si) in enumerate(proba):
        print(f"  Vehículo {i}: P(no) = {p_no:.4f} | P(si) = {p_si:.4f}")
    print()
```

3. Resumen general del proceso

El proceso inicia abordando un problema de clasificación donde se tienen datos de vehículos clasificados en múltiples categorías y se quiere usar Regresión Logística para predecir la categoría de autos nuevos. El reto principal es que la Regresión Logística clásica solo puede decidir entre dos opciones y no entre cuatro.

Para solucionarlo se emplea la estrategia One-vs-Rest: se entrena un modelo que responde si un vehículo pertenece a una clase específica o no, y se repite para todas las categorías. La base matemática está respaldada por el archivo Excel, donde cada modelo calcula un peso para cada atributo, como velocidad o carga.

Finalmente, en la etapa de predicción, cuando llegan vehículos nuevos, se evalúan por todos los modelos. Cada modelo emite un porcentaje de probabilidad y el vehículo se etiqueta con la clase cuyo modelo arrojó el valor más alto.

Proyectos

Universidad Autónoma de Tamaulipas

Ingeniería en Ciencia de Datos e Inteligencia Artificial

Unidad: Fundamentos de Machine Learning

Comparación de los algoritmos KNN y Regresión Logística en la clasificación del género de videojuegos

Proyecto académico de clasificación supervisada

FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109

70 Aniversario Facultad de Ingeniería Tampico 1956-2026

Estudiantes: Juan Carlos Méndez Gómez
Gianncarlo Martinez
Juan Carlos Banda Hernandez
Grado y Grupo: 3 R
Docente: Garcia Ruiz Alejandro Humberto
Fecha: 12 de octubre de 2026

Introducción

La clasificación supervisada consiste en asignar una categoría a un caso nuevo a partir de ejemplos previos cuya categoría es conocida. En este proyecto se aborda un problema de clasificación multiclase: determinar el género de un videojuego (Acción, Deportes, Estrategia o RPG) a partir de cinco atributos numéricos que lo describen.

Se comparan dos algoritmos estudiados en la unidad 2 de Fundamentos de Machine Learning: **K vecinos más cercanos (KNN)**, un método basado en instancias que clasifica por votación de los casos de entrenamiento más próximos, y la **Regresión Logística**, un modelo paramétrico que estima coeficientes y calcula una puntuación por clase. Ambos se implementaron en Python y se verificaron mediante cálculos manuales en Excel, y su desempeño se mide con la tasa de acierto (exactitud) sobre un mismo conjunto de prueba.

Objetivo

Objetivo general

Comparar el desempeño de los algoritmos KNN y Regresión Logística en la clasificación del género de videojuegos, utilizando la exactitud como métrica principal.

Objetivos específicos

- Describir los datos utilizados y su división en conjuntos de entrenamiento y prueba.
- Explicar la preparación de los datos y la implementación de ambos algoritmos en Python.
- Documentar los cálculos manuales realizados en Excel para cada algoritmo.
- Calcular la exactitud de cada modelo sobre los casos de prueba.
- Contrastar los resultados e identificar ventajas y desventajas de cada algoritmo.

Descripción de los datos

Archivos analizados

Se revisaron los siguientes archivos (Tabla 1).

Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

Tabla 1: Archivos analizados

Archivo	Contenido
`Dataset_Videojuegos_ML` (`.csv`, `.xlsx`)	Dataset completo: 100 registros.
`Entrenamiento_v` (`.csv`, `.xlsx`)	Conjunto de entrenamiento: 80 registros.
`Prueba_v` (`.csv`, `.xlsx`)	Conjunto de prueba: 20 registros.
`KNN_Manual_videojuegos.py`	Implementación de KNN en Python.
`RegresionLogicaVideojuegos.py`	Implementación de Regresión Logística en Python.
`KNN_videojuegos.xlsx`	Cálculo manual de KNN (hojas *Prueba*, *Entrenamiento*, *Distancias*, *Posicion* y *Ordenado*).
`Actividad_ML_videojuegos.xlsx`	Cálculo manual de Regresión Logística (hoja *Regresión logística*).

Variables

Cada registro describe un videojuego mediante cinco atributos numéricos enteros y su género. El dataset no contiene valores nulos. La Tabla 2 resume las variables y el rango observado en el dataset completo.

Tabla 2: Variables del dataset (rango observado en los 100 registros)

Variable	Tipo	Rango observado
`Intensidad_Accion`	Atributo numérico	1–100
`Complejidad_Historia`	Atributo numérico	1–100
`Necesidad_Estrategia`	Atributo numérico	1–100
`Habilidad_Requerida`	Atributo numérico	26–100
`Competitividad`	Atributo numérico	1–100
`Genero_Videojuego`	Clase (4 categorías)	Acción, Deportes, Estrategia, RPG

División en entrenamiento y prueba

El dataset completo está balanceado: contiene 25 registros de cada género. El conjunto de entrenamiento (80 registros) y el de prueba (20 registros) suman exactamente los 100 registros originales; en el archivo se observa que el entrenamiento corresponde a las primeras 80 filas y la prueba a las últimas 20. La distribución por género se muestra en la Tabla 3.

Tabla 3: Distribución de registros por género y conjunto

Género	Dataset completo	Entrenamiento	Prueba
Acción	25	19	6
Deportes	25	19	6
Estrategia	25	19	6
RPG	25	23	2
Total	**100**	**80**	**20**

Perfil promedio por género

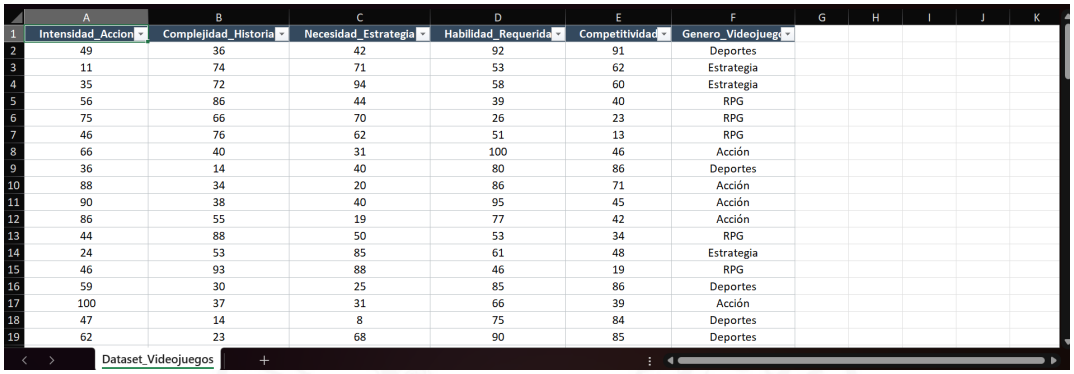
La Tabla 4 presenta el promedio de cada atributo por género en el conjunto de entrenamiento. Cada género destaca en un atributo distinto: Acción en `Intensidad_Accion`, Estrategia en `Necesidad_Estrategia`, RPG en `Complejidad_Historia` y Deportes en `Competitividad`. Acción y Deportes comparten valores altos de `Habilidad_Requerida`.

FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109
70 Aniversario Facultad de Ingeniería Tampico 1956-2026

Tabla 4: Promedio de los atributos por género (conjunto de entrenamiento, $n = 80$)

Género	Int.	Hist.	Estr.	Hab.	Comp.
Acción	84.7	39.6	31.5	77.7	44.6
Deportes	56.9	18.9	34.7	85.0	89.0
Estrategia	26.3	55.2	86.1	63.2	60.1
RPG	43.1	84.3	61.3	49.7	33.4

Int. = `Intensidad_Accion`; Hist. = `Complejidad_Historia`; Estr. = `Necesidad_Estrategia`; Hab. = `Habilidad_Requerida`; Comp. = `Competitividad`.



	A	B	C	D	E	F
1	Intensidad_Accion	Complejidad_Historia	Necesidad_Estrategia	Habilidad_Requerida	Competitividad	Genero_Videojuego
2	49	36	42	92	91	Deportes
3	11	74	71	53	62	Estrategia
4	35	72	94	58	60	Estrategia
5	56	86	44	39	40	RPG
6	75	66	70	26	23	RPG
7	46	76	62	51	13	RPG
8	66	40	31	100	46	Acción
9	36	14	40	80	86	Deportes
10	88	34	20	86	71	Acción
11	90	38	40	95	45	Acción
12	86	55	19	77	42	Acción
13	44	88	50	53	34	RPG
14	24	53	85	61	48	Estrategia
15	46	93	88	46	19	RPG
16	59	30	25	85	86	Deportes
17	100	37	31	66	39	Acción
18	47	14	8	75	84	Deportes
19	62	23	68	90	85	Deportes

Dataset_Videojuegos

Figura 3: Muestra del dataset de videojuegos.

Metodología

Flujo general

El procedimiento aplicado a ambos algoritmos fue el siguiente:

1. Lectura de los conjuntos de entrenamiento y prueba a partir de los archivos `.csv`.
2. Ajuste de cada modelo con los 80 registros de entrenamiento.
3. Predicción del género de los 20 casos de prueba.
4. Comparación de la clase predicha con la clase real de cada caso.
5. Cálculo de la exactitud y verificación manual de los resultados en Excel.

Preparación de los datos

La preparación de los datos fue mínima y se describe a continuación:

- **Lectura:** los archivos `.csv` se cargan con `pandas` usando la codificación `utf-8-sig`, que gestiona la marca de orden de bytes presente al inicio de los archivos.
- **Separación de variables:** los cinco atributos se utilizan como variables predictoras y `Genero_Videojuego` como clase.
- **Valores nulos:** no se encontraron valores nulos en los datos, por al que no fue necesaria ninguna imputación.
- **Escalamiento:** los scripts no aplican normalización ni estandarización. Los cinco atributos están expresados en una escala similar (aproximadamente de 1 a 100).
- **Regresión Logística:** para cada género se genera una copia de los datos de entrenamiento en la que la clase se recodifica como «sí» (1) si el registro pertenece a ese género y «no» (0) en caso contrario, de modo que se obtiene un problema binario por género (estrategia *uno contra todos*). Las etiquetas se limpian de espacios sobrantes (`strip` en Python y `TRIM` en Excel).

Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico
VERDAD, BELLEZA, PROBIDAD
FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109
70 Aniversario Facultad de Ingeniería Tampico 1956-2026

Universidad Autónoma de TAMAULIPAS
Facultad de Ingeniería Tampico

Métrica de evaluación

La métrica principal es la tasa de acierto o exactitud:

$$\text{Exactitud} = \frac{\text{aciertos}}{\text{total_pruebas}} \tag{1}$$

donde *aciertos* es el número de casos de prueba cuya clase predicha coincide con la clase real y *total_pruebas* es el número de casos de prueba, que en este proyecto es 20.

Nota terminológica: el script de KNN imprime este valor con la etiqueta «Precision», y las hojas de Excel lo denominan «Precisión». En todos los casos el cálculo realizado es aciertos entre total de pruebas, es decir, la exactitud definida arriba.

Herramientas

Se utilizó Python con las bibliotecas `pandas`, `statistics` y `scikit-learn`, además de Excel para los cálculos manuales. Las versiones de Python y de las bibliotecas no están registradas en los archivos, por lo que no pudieron determinarse.

Implementación de KNN

Fundamento

KNN no ajusta parámetros durante el entrenamiento: conserva los registros de entrenamiento y, para cada caso nuevo $\mathbf{c}$, calcula su distancia euclidiana a todos ellos:

$$d(\mathbf{x}_i, \mathbf{c}) = \sqrt{\sum_{j=1}^{5} (x_{ij} - c_j)^2} \tag{2}$$

Después ordena los registros de menor a mayor distancia, selecciona los K más cercanos y asigna la clase más frecuente entre ellos (la moda). En este proyecto, K se define como la parte entera de la raíz cuadrada del número de registros de entrenamiento N:

$$K = \lfloor \sqrt{N} \rfloor = \lfloor \sqrt{80} \rfloor = 8 \tag{3}$$

Implementación en Python

El script `KNN_Manual_videojuegos.py` implementa el algoritmo de forma manual con `pandas`, sin utilizar el clasificador de `scikit-learn`. Para cada uno de los 20 casos de prueba:

1. Calcula la distancia euclidiana contra los 80 registros de entrenamiento en las cinco dimensiones.
2. Ordena los registros de menor a mayor distancia (ordenamiento estable, de modo que en caso de empate se respeta el orden del archivo).
3. Se queda con los $K = 8$ vecinos más cercanos.
4. Asigna como clase predicha la moda de las clases de esos vecinos y la compara con la clase real.

Al final, el script imprime el número de aciertos y la exactitud. El siguiente es un fragmento abreviado del script:

```
N = len(df)
K = int(N ** 0.5)
...
for i, caso in prueba.iterrows():
    df["Distancia"] = ((df[atributos] - caso[atributos]) ** 2).sum(axis=1) ** 0.5
```

FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109
70 Aniversario Facultad de Ingeniería Tampico 1956-2026

```
6       df_ordenado = df.sort_values(by="Distancia", kind="stable")
7       k_vecinos = df_ordenado.head(K)
8       clase_predicha = statistics.mode(k_vecinos[columna_clase])
```

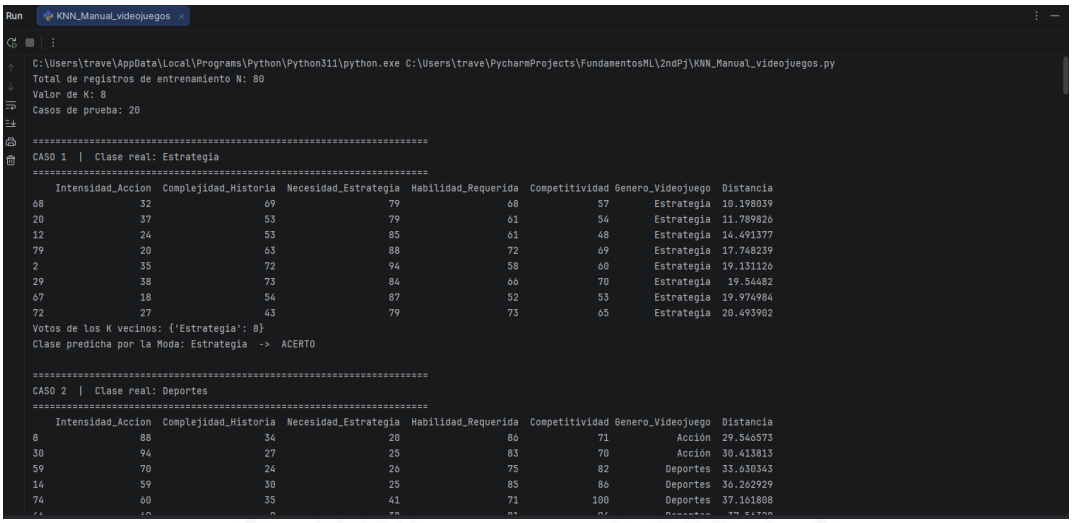


Figura 4: Salida del script de KNN.

FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109



Figura 5: Salida del script de KNN.

Cálculos realizados en Excel

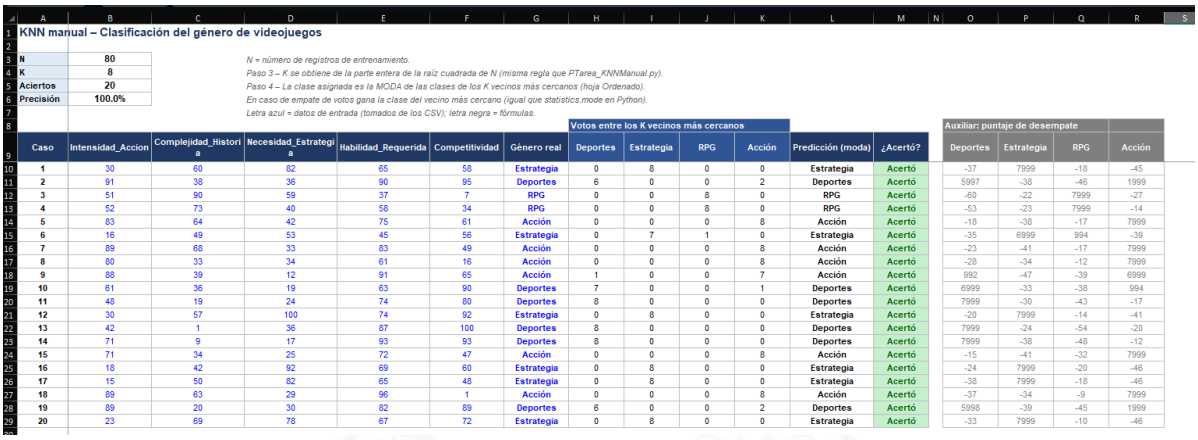
El archivo `KNN_videojuegos.xlsx` reproduce paso a paso el algoritmo con fórmulas de Excel, aplicando la misma regla para K que el script de Python. Sus hojas son:

- **Entrenamiento:** los 80 registros de entrenamiento con sus cinco atributos y su clase.
- **Distancias:** distancia euclidiana de cada uno de los 20 casos de prueba contra los 80 registros (fórmula `SQRT(POWER(...))`), lo que da $80 \times 20 = 1600$ distancias.
- **Posicion:** posición de cada registro (1 = más cercano) para cada caso, calculada con `SUMPRODUCT`; los empates de distancia se resuelven por el orden del archivo.
- **Ordenado:** registros ordenados por distancia para cada caso, con los K vecinos más cercanos sombreados.
- **Prueba:** calcula $N = 80$ (`COUNT`) y $K = 8$ (`INT(SQRT(N))`); cuenta los votos de cada género entre los K vecinos (`COUNTIFS`); asigna la predicción por mayoría; indica si el caso se acertó; y calcula los aciertos y

Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

la exactitud. Incluye un puntaje auxiliar de desempate: si dos géneros empatan en votos, gana el del vecino más cercano.

En ninguno de los 20 casos hubo empate de votos: el menor número de votos obtenido por la clase ganadora fue 6 de 8.



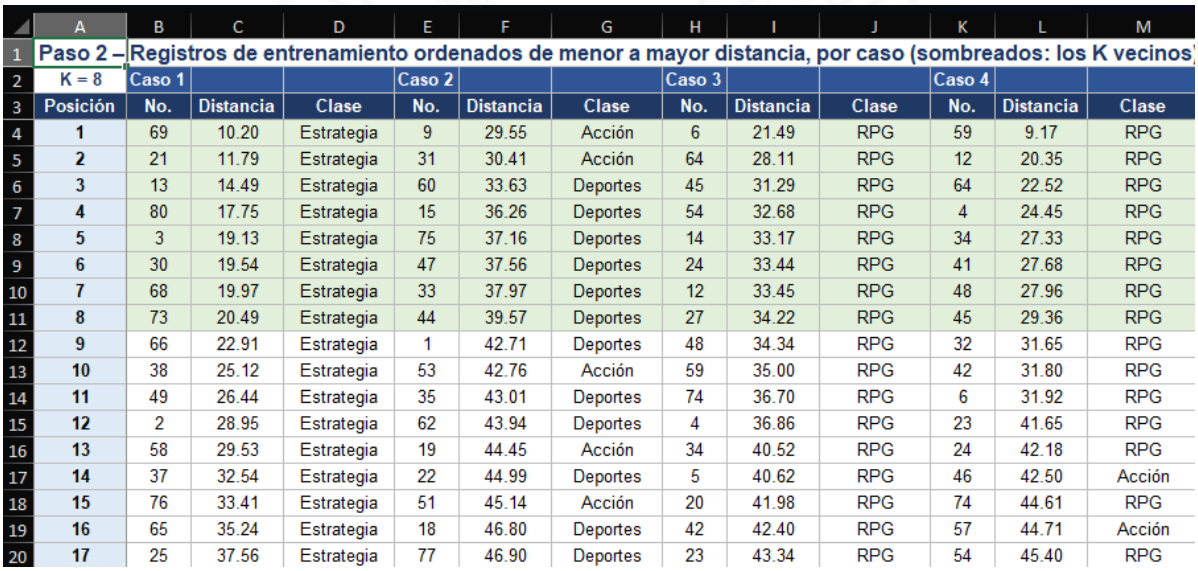
KNN manual – Clasificación del género de videojuegos

N	80
K	8
Aciertos	20
Precisión	100.0%

N = número de registros de entrenamiento.
Paso 3 – K se obtiene de la parte entera de la raíz cuadrada de N (misma regla que PTarea_KNNManual.py).
Paso 4 – La clase asignada es la MODA de las clases de los K vecinos más cercanos (hoja Ordenado).
En caso de empate de votos gana la clase del vecino más cercano (igual que statistics.mode en Python).
Letra azul = datos de entrada (tomados de los CSV); letra negra = fórmulas.

							Votos entre los K vecinos más cercanos						Auxiliar: puntaje de desempate			
Caso	Intensidad_Accion	Complejidad_Historia	Necesidad_Estrategia	Habilidad_Requerida	Competitividad	Género real	Deportes	Estrategia	RPG	Acción	Predicción (moda)	¿Acertó?	Deportes	Estrategia	RPG	Acción
1	30	60	82	65	58	Estrategia	0	8	0	0	Estrategia	Acertó	-37	7999	-18	-45
2	91	38	36	90	95	Deportes	6	0	0	2	Deportes	Acertó	5997	-38	-46	1999
3	51	90	59	37	7	RPG	0	0	8	0	RPG	Acertó	-60	-22	7999	-27
4	52	73	40	58	34	RPG	0	0	8	0	RPG	Acertó	-53	-23	7999	-14
5	83	64	42	75	61	Acción	0	0	0	8	Acción	Acertó	-18	-38	-17	7999
6	16	49	53	45	56	Estrategia	0	7	1	0	Estrategia	Acertó	-35	6999	994	-39
7	89	68	33	83	48	Acción	0	0	0	8	Acción	Acertó	-23	-41	-17	7999
8	80	33	34	61	16	Acción	0	0	0	8	Acción	Acertó	-28	-34	-12	7999
9	88	39	12	91	65	Acción	1	0	0	7	Acción	Acertó	992	-47	-39	6999
10	61	36	19	63	90	Deportes	7	0	0	1	Deportes	Acertó	6999	-33	-38	994
11	48	19	24	74	80	Deportes	8	0	0	0	Deportes	Acertó	7999	-30	-43	-17
12	30	57	100	74	92	Estrategia	0	8	0	0	Estrategia	Acertó	-20	7999	-14	-41
13	42	1	36	87	100	Deportes	8	0	0	0	Deportes	Acertó	7999	-24	-54	-20
14	71	9	17	93	93	Deportes	8	0	0	0	Deportes	Acertó	7999	-38	-48	-12
15	71	34	25	72	47	Acción	0	0	0	8	Acción	Acertó	-15	-41	-32	7999
16	18	42	92	69	60	Estrategia	0	8	0	0	Estrategia	Acertó	-24	7999	-20	-46
17	15	50	82	65	48	Estrategia	0	8	0	0	Estrategia	Acertó	-38	7999	-18	-46
18	89	63	29	96	1	Acción	0	0	0	8	Acción	Acertó	-37	-34	-9	7999
19	89	20	30	82	89	Deportes	6	0	0	2	Deportes	Acertó	5998	-39	-45	1999
20	23	69	78	67	72	Estrategia	0	8	0	0	Estrategia	Acertó	-33	7999	-10	-46

Figura 6: Hoja «Prueba» del cálculo manual de KNN.



Paso 2 – Registros de entrenamiento ordenados de menor a mayor distancia, por caso (sombreados: los K vecinos)

K = 8	Caso 1			Caso 2			Caso 3			Caso 4		
Posición	No.	Distancia	Clase	No.	Distancia	Clase	No.	Distancia	Clase	No.	Distancia	Clase
1	69	10.20	Estrategia	9	29.55	Acción	6	21.49	RPG	59	9.17	RPG
2	21	11.79	Estrategia	31	30.41	Acción	64	28.11	RPG	12	20.35	RPG
3	13	14.49	Estrategia	60	33.63	Deportes	45	31.29	RPG	64	22.52	RPG
4	80	17.75	Estrategia	15	36.26	Deportes	54	32.68	RPG	4	24.45	RPG
5	3	19.13	Estrategia	75	37.16	Deportes	14	33.17	RPG	34	27.33	RPG
6	30	19.54	Estrategia	47	37.56	Deportes	24	33.44	RPG	41	27.68	RPG
7	68	19.97	Estrategia	33	37.97	Deportes	12	33.45	RPG	48	27.96	RPG
8	73	20.49	Estrategia	44	39.57	Deportes	27	34.22	RPG	45	29.36	RPG
9	66	22.91	Estrategia	1	42.71	Deportes	48	34.34	RPG	32	31.65	RPG
10	38	25.12	Estrategia	53	42.76	Acción	59	35.00	RPG	42	31.80	RPG
11	49	26.44	Estrategia	35	43.01	Deportes	74	36.70	RPG	6	31.92	RPG
12	2	28.95	Estrategia	62	43.94	Deportes	4	36.86	RPG	23	41.65	RPG
13	58	29.53	Estrategia	19	44.45	Acción	34	40.52	RPG	24	42.18	RPG
14	37	32.54	Estrategia	22	44.99	Deportes	5	40.62	RPG	46	42.50	Acción
15	76	33.41	Estrategia	51	45.14	Acción	20	41.98	RPG	74	44.61	RPG
16	65	35.24	Estrategia	18	46.80	Deportes	42	42.40	RPG	57	44.71	Acción
17	25	37.56	Estrategia	77	46.90	Deportes	23	43.34	RPG	54	45.40	RPG

Figura 7: Distancias y vecinos más cercanos en Excel.

Implementación de Regresión Logística

Fundamento

La regresión logística binaria modela la probabilidad de que un registro pertenezca a una clase mediante la función sigmoide aplicada a una combinación lineal de los atributos:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-z}}, \qquad z = \beta_0 + \sum_{j=1}^{5} \beta_j x_j \tag{4}$$

FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109
70 Aniversario Facultad de Ingeniería Tampico 1956-2026

Como el problema tiene cuatro géneros, se entrena un modelo por género con la estrategia *uno contra todos* («género contra el resto»). Para clasificar un caso nuevo se evalúan los cuatro modelos y se asigna el género con mayor puntuación.

Implementación en Python

El script `Regresionlogicamulticlase_videojuegos.py` entrena, con los 80 registros de `Entrenamiento_v.csv`, un modelo `LogisticRegression` de `scikit-learn` por cada género. Solo se modifica el parámetro `max_iter=1000`; el resto de hiperparámetros conserva los valores por defecto de la biblioteca (incluida su regularización por defecto). El siguiente es un fragmento del script:

```
for clase in clases_unicas:
    sub_df = df.copy()
    sub_df[columna_clase] = sub_df[columna_clase].apply(
        lambda x: "si" if str(x).strip() == clase else "no")
    sub_dataframes[clase] = sub_df

    X = sub_df[atributos].values
    y = (sub_df[columna_clase] == "si").astype(int).values  # si=1, no=0

    modelo = LogisticRegression(max_iter=1000)
    modelo.fit(X, y)
    modelos[clase] = modelo
```

La predicción se realiza seleccionando el modelo con mayor probabilidad de la clase «sí»:

```
probabilidades = pd.DataFrame(
    {c: m.predict_proba(nuevos[atributos].values)[:, 1] for c, m in modelos.items()})
nuevos["Genero_predicho"] = probabilidades.idxmax(axis=1)
```

Los 20 casos de `Prueba_v.csv` se cargan en `nuevos`, y el script compara el género predicho con el género real de cada caso. Al ejecutarlo, imprime la predicción de cada caso, el total de aciertos y la exactitud. Los resultados se presentan en la Tabla 6.

Los coeficientes estimados por el script se presentan en la Tabla 5. Coinciden, redondeados a cuatro decimales, con los coeficientes registrados en la hoja de Excel.

Tabla 5: Coeficientes de los cuatro modelos «género contra el resto»

Modelo	β_0	β_1	β_2	β_3	β_4	β_5
Deportes	−0,0033	−0,2302	−0,4321	−0,1322	0,1264	0,3100
Estrategia	−29,3562	−0,4732	−0,1996	0,6587	0,1547	−0,0023
RPG	−23,6217	0,1060	0,7172	−0,0538	−0,3034	−0,2194
Acción	−60,7067	0,7767	0,1117	−0,3693	0,3449	−0,2503

β_0 = intercepto; β_1 = `Intensidad_Accion`; β_2 = `Complejidad_Historia`; β_3 = `Necesidad_Estrategia`; β_4 = `Habilidad_Requerida`; β_5 = `Competitividad`.

En cada modelo, el mayor coeficiente positivo corresponde al atributo que caracteriza al género: `Competitividad` en Deportes, `Necesidad_Estrategia` en Estrategia, `Complejidad_Historia` en RPG e `Intensidad_Accion` en Acción.

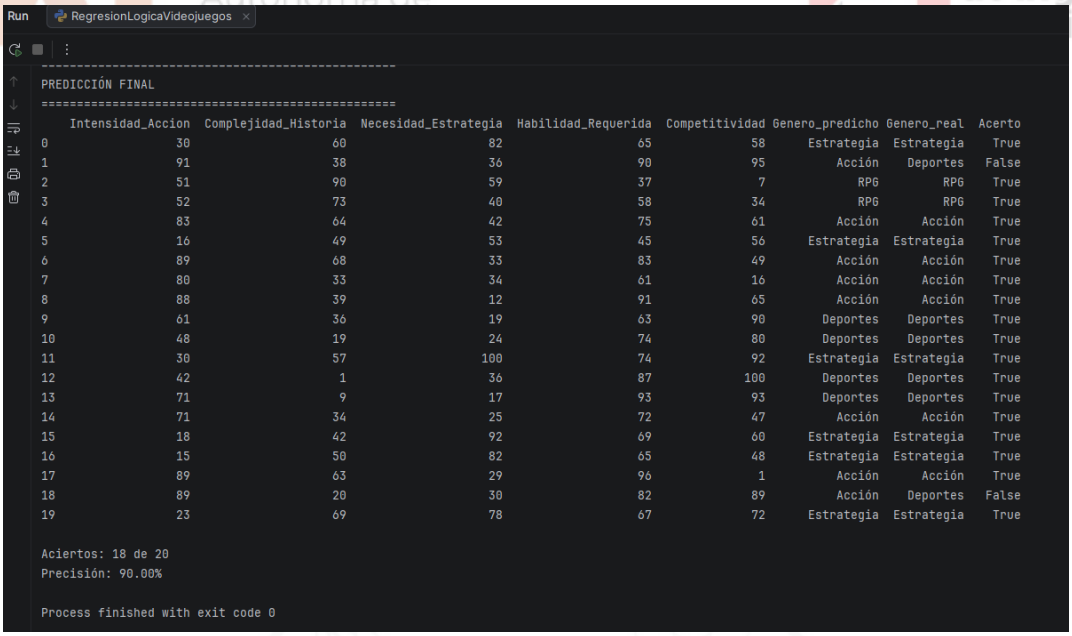


Figura 8: Salida del script de Regresión Logística.

Cálculos realizados en Excel

La hoja *Regresión logística* del archivo `Actividad_ML_videojuegos.xlsx` aplica manualmente el modelo a los 20 casos de prueba. Los coeficientes β se ingresan como constantes (con cuatro decimales); la hoja no los estima. Para cada caso existe un bloque de 13 filas con los siguientes cálculos, repetidos para cada uno de los cuatro géneros:

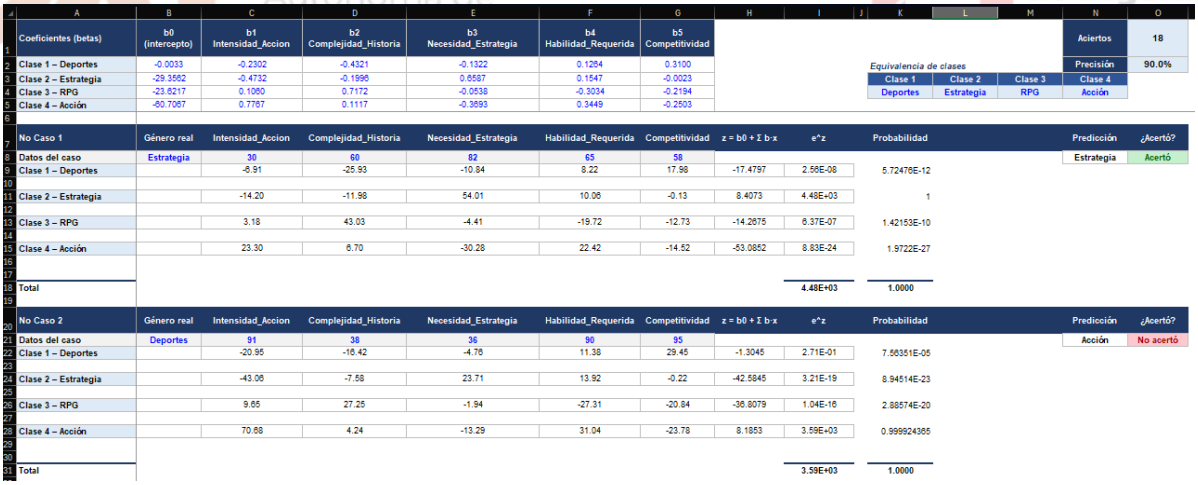
1. Producto de cada atributo por su coeficiente ($\beta_j x_j$).
2. Puntuación lineal $z = \beta_0 + \sum_j \beta_j x_j$.
3. Exponencial e^z, calculada con la aproximación $e \approx 2{,}7182$.
4. Probabilidad normalizada:

$$p_c = \frac{e^{z_c}}{\sum_{k=1}^{4} e^{z_k}} \tag{5}$$

5. Predicción: el género con mayor probabilidad, y verificación contra el género real (columna «¿Acertó?»).

En las celdas superiores de la hoja se cuentan los aciertos (`COUNTIF`) y se calcula la exactitud como aciertos entre total de casos.

La normalización de la hoja difiere de las probabilidades individuales del script de Python (que son una sigmoide por modelo), pero ambas crecen de forma monótona con z, por lo que el género con mayor puntuación es el mismo en los dos enfoques.



Coeficientes (betas)	b0 (intercepto)	b1 Intensidad_Accion	b2 Complejidad_Historia	b3 Necesidad_Estrategia	b4 Habilidad_Requerida	b5 Competitividad
Clase 1 – Deportes	-0.0033	-0.2302	-0.4321	-0.1322	0.1264	0.3100
Clase 2 – Estrategia	-29.3562	-0.4732	-0.1996	0.6587	0.1547	-0.0023
Clase 3 – RPG	-23.6217	0.1060	0.7172	-0.0538	-0.3034	-0.2194
Clase 4 – Acción	-60.7067	0.7767	0.1117	-0.3693	0.3449	-0.2503

Aciertos	18
Precisión	90.0%

Equivalencia de clases

Clase 1	Clase 2	Clase 3	Clase 4
Deportes	Estrategia	RPG	Acción

No Caso 1	Género real	Intensidad_Accion	Complejidad_Historia	Necesidad_Estrategia	Habilidad_Requerida	Competitividad	z = b0 + Σ b·x	e^z	Probabilidad
Datos del caso	Estrategia	30	60	82	65	58			
Clase 1 – Deportes		-6.91	-25.93	-10.84	8.22	17.96	-17.4797	2.56E-08	5.72476E-12
Clase 2 – Estrategia		-14.20	-11.98	54.01	10.06	-0.13	8.4073	4.48E+03	1
Clase 3 – RPG		3.18	43.03	-4.41	-19.72	-12.73	-14.2675	6.37E-07	1.42163E-10
Clase 4 – Acción		23.30	6.70	-30.28	22.42	-14.52	-53.0852	8.83E-24	1.9722E-27
Total								4.48E+03	1.0000

Predicción	¿Acertó?
Estrategia	Acertó

No Caso 2	Género real	Intensidad_Accion	Complejidad_Historia	Necesidad_Estrategia	Habilidad_Requerida	Competitividad	z = b0 + Σ b·x	e^z	Probabilidad
Datos del caso	Deportes	91	38	36	90	95			
Clase 1 – Deportes		-20.95	-16.42	-4.76	11.38	29.45	-1.3045	2.71E-01	7.56351E-05
Clase 2 – Estrategia		-43.06	-7.58	23.71	13.92	-0.22	-42.5845	3.21E-19	8.94514E-23
Clase 3 – RPG		9.65	27.25	-1.94	-27.31	-20.84	-36.8079	1.04E-16	2.88574E-20
Clase 4 – Acción		70.68	4.24	-13.29	31.04	-23.78	8.1853	3.59E+03	0.999924365
Total								3.59E+03	1.0000

Predicción	¿Acertó?
Acción	No acertó

Figura 9: Cálculo manual de Regresión Logística en Excel.

Resultados

KNN

Con $N = 80$ registros de entrenamiento y $K = 8$, el modelo clasificó correctamente los 20 casos de prueba. Los resultados del script de Python y del cálculo en Excel coinciden:

$$\text{Exactitud}_{\text{KNN}} = \frac{20}{20} = 1{,}00\ (100\ \%)$$

En 15 de los 20 casos los 8 vecinos pertenecían a la clase predicha; en los 5 restantes la clase ganadora obtuvo 7 votos (casos 6, 9 y 10) o 6 votos (casos 2 y 19).

Regresión Logística

El script de Python y el cálculo en Excel, ambos con los coeficientes de la Tabla 5, acertaron 18 de los 20 casos de prueba:

$$\text{Exactitud}_{\text{RL}} = \frac{18}{20} = 0{,}90 \quad (90\,\%)$$

Los dos errores corresponden a los casos 2 y 19, ambos de género Deportes clasificados como Acción:

- **Caso 2** (atributos 91, 38, 36, 90, 95): puntuación $z = -1{,}3045$ para Deportes y $z = 8{,}1853$ para Acción; probabilidad normalizada de Acción de 0,9999.
- **Caso 19** (atributos 89, 20, 30, 82, 89): puntuación $z = 4{,}8557$ para Deportes y $z = 5{,}5797$ para Acción; probabilidades normalizadas de 0,3265 y 0,6735, respectivamente. La diferencia entre ambas puntuaciones es de solo 0,72, por lo que la clasificación estuvo cerca de la frontera de decisión.

Resultados por caso de prueba

La Tabla 6 muestra la predicción de cada modelo para los 20 casos de prueba, junto con el número de votos que obtuvo la clase predicha por KNN entre los 8 vecinos.

Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

Tabla 6: Resultados por caso de prueba

Caso	Género real	KNN	Votos	Reg. Logística	¿KNN acertó?	¿RL acertó?
1	Estrategia	Estrategia	8/8	Estrategia	Sí	Sí
2	Deportes	Deportes	6/8	Acción	Sí	**No**
3	RPG	RPG	8/8	RPG	Sí	Sí
4	RPG	RPG	8/8	RPG	Sí	Sí
5	Acción	Acción	8/8	Acción	Sí	Sí
6	Estrategia	Estrategia	7/8	Estrategia	Sí	Sí
7	Acción	Acción	8/8	Acción	Sí	Sí
8	Acción	Acción	8/8	Acción	Sí	Sí
9	Acción	Acción	7/8	Acción	Sí	Sí
10	Deportes	Deportes	7/8	Deportes	Sí	Sí
11	Deportes	Deportes	8/8	Deportes	Sí	Sí
12	Estrategia	Estrategia	8/8	Estrategia	Sí	Sí
13	Deportes	Deportes	8/8	Deportes	Sí	Sí
14	Deportes	Deportes	8/8	Deportes	Sí	Sí
15	Acción	Acción	8/8	Acción	Sí	Sí
16	Estrategia	Estrategia	8/8	Estrategia	Sí	Sí
17	Estrategia	Estrategia	8/8	Estrategia	Sí	Sí
18	Acción	Acción	8/8	Acción	Sí	Sí
19	Deportes	Deportes	6/8	Acción	Sí	**No**
20	Estrategia	Estrategia	8/8	Estrategia	Sí	Sí

La Tabla 7 desglosa los aciertos por género y la Tabla 8 resume la exactitud de ambos modelos.

Tabla 7: Aciertos por género en el conjunto de prueba

Género	Casos de prueba	Aciertos KNN	Aciertos RL
Acción	6	6	6
Deportes	6	6	4
Estrategia	6	6	6
RPG	2	2	2
Total	**20**	**20**	**18**

Tabla 8: Exactitud de los modelos en el conjunto de prueba

Modelo	Aciertos	Total de pruebas	Exactitud
KNN ($K = 8$)	20	20	1.00 (100 %)
Regresión Logística	18	20	0.90 (90 %)

VERDAD, BELLEZA, PROBIDAD
FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109
70 Aniversario Facultad de Ingeniería Tampico 1956-2026

Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

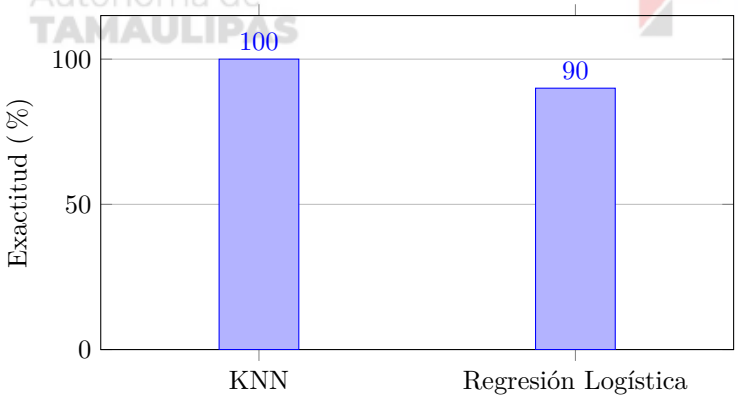


Figura 10: Exactitud de KNN y Regresión Logística en los 20 casos de prueba.

Comparación de modelos

Desempeño

En el conjunto de prueba, KNN obtuvo una exactitud de 100 % (20 de 20) y la Regresión Logística de 90 % (18 de 20), es decir, una diferencia de dos casos. Ambos modelos acertaron todos los casos de Acción, Estrategia y RPG; la diferencia se concentra por completo en el género Deportes, donde KNN acertó 6 de 6 y la Regresión Logística 4 de 6.

La Tabla 9 muestra el promedio de cada atributo por género en los datos de entrenamiento. Deportes y Acción tienen valores altos de `Habilidad_Requerida` (85.0 y 77.7), lo que podría dificultar que una frontera lineal separe ambos géneros. Esta es una interpretación de los promedios, no una prueba: no se evaluaron otras fronteras ni otras variables.

Tabla 9: Promedio de cada atributo por género en los datos de entrenamiento

Género	Intensidad	Complejidad	Estrategia	Habilidad	Competitividad
Acción	84.7	39.6	31.5	77.7	44.6
Deportes	56.9	18.9	34.7	85.0	89.0
Estrategia	26.3	55.2	86.1	63.2	60.1
RPG	43.1	84.3	61.3	49.7	33.4

Estos resultados deben interpretarse con cautela: con 20 casos de prueba cada caso equivale a 5 puntos porcentuales, y solo se evaluó una partición de los datos.

VERDAD, BELLEZA, PROBIDAD
FACULTAD DE INGENIERÍA TAMPICO
Centro Universitario Sur
Tampico (833) 241-20-00, ext. 3451
C.P. 89109
70 Aniversario Facultad de Ingeniería Tampico 1956-2026

Perspectiva práctica

Tabla 10: Comparación práctica entre KNN y Regresión Logística en este proyecto

Criterio	KNN	Regresión Logística
Entrenamiento	No ajusta parámetros; conserva los 80 registros de entrenamiento.	Estima los coeficientes de cuatro modelos «género contra el resto» (6 coeficientes cada uno).
Predicción de un caso	Calcula 80 distancias, las ordena y vota entre los $K = 8$ vecinos.	Evalúa cuatro funciones lineales (una por género) y elige la de mayor puntuación.
Hiperparámetros	K (aquí, $\lfloor\sqrt{N}\rfloor$) y la distancia euclidiana.	`max_iter`=1000; el resto, valores por defecto de `scikit-learn`.
Interpretación	Cada predicción se justifica con sus vecinos y sus votos (por ejemplo, 6 de 8 en el caso 2).	Se interpretan el signo y la magnitud de los coeficientes (Tabla 5).
Cálculo manual en Excel	Extenso: distancias, posiciones, ordenamiento y votos en cinco hojas.	Operaciones sencillas por caso (productos, suma, exponencial y normalización), pero parte de coeficientes ya estimados que la hoja no calcula.
Exactitud obtenida	100 % (20 de 20)	90 % (18 de 20)

Ventajas y desventajas

KNN Ventajas

- Concepto simple e intuitivo, fácil de implementar y de verificar manualmente.
- No requiere una fase de entrenamiento ni supone una forma específica para la frontera de decisión.
- Maneja de forma natural problemas con más de dos clases.
- Sus predicciones son explicables caso por caso mediante los vecinos y los votos.

Desventajas

- Debe conservar todos los datos de entrenamiento y calcular las distancias a todos ellos en cada predicción, por lo que el costo crece con el tamaño del conjunto.
- Es sensible a la escala de los atributos y a la presencia de atributos poco relevantes.
- Su resultado depende de la elección de K y de la métrica de distancia.
- No produce un modelo compacto ni coeficientes que resuman la relación entre atributos y clase.

Regresión Logística Ventajas

- Genera un modelo compacto (los coeficientes) que permite predecir rápidamente nuevos casos.
- Los coeficientes son interpretables: indican qué atributos favorecen cada género.
- Entrega puntuaciones y probabilidades, útiles para evaluar qué tan segura es una predicción.
- Es un modelo base ampliamente utilizado y fácil de extender.

Desventajas

- Cada modelo separa las clases mediante una frontera lineal, lo que limita su flexibilidad si las clases no son separables de ese modo.
- Para más de dos clases requiere una estrategia adicional (aquí, uno contra todos).

Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

- Requiere una fase de optimización para estimar los coeficientes, con parámetros como el número máximo de iteraciones.

Conclusiones

- En el conjunto de prueba de 20 casos, KNN con $K = 8$ obtuvo una exactitud de 100 % (20 aciertos), mientras que la Regresión Logística obtuvo 90 % (18 aciertos).
- Los resultados de KNN coinciden entre el script de Python y el cálculo manual en Excel, y los resultados de la Regresión Logística coinciden entre el script de Python y la hoja de Excel.
- La diferencia de desempeño se debe únicamente a dos casos de género Deportes (casos 2 y 19) que la Regresión Logística clasificó como Acción; los demás casos se clasificaron correctamente con ambos modelos.
- Desde la perspectiva práctica, KNN es simple y no requiere entrenamiento, pero debe calcular distancias contra todo el conjunto de entrenamiento en cada predicción; la Regresión Logística produce un modelo compacto e interpretable, aunque con fronteras lineales.
- Dado el tamaño reducido del conjunto de prueba (20 casos, con solo 2 de RPG) y la evaluación sobre una única partición, la superioridad de KNN observada aquí no puede generalizarse sin evidencia adicional.
- Como trabajo futuro se sugiere probar otros valores de K y aplicar validación cruzada para comparar ambos modelos con más casos.

VERDAD, BELLEZA, PROBIDAD